

# Towards a Semantics-Preserving Transformation from UML/OCL Models to Property Graphs

Technical Report: Complete Formal Interface, Proof Obligations, and Evidence  
Boundary

Anonymous Authors

Technical report accompanying the main paper

## Abstract

This report specifies a semantics-preserving transformation from a restricted UML/OCL validation profile to a property graph and a typed Cypher query, with correctness stated as equality of stable violation-ID sets. It makes explicit the admitted language, typed semantic domains, graph-adequacy clauses, normalization and realization contracts, theorem assumptions, proof dependencies, and evidence boundary. The result is deliberately conditional: constructor-local realization and selected backend-semantic agreements remain premises unless separately discharged. Formal derivations and finite observations support particular obligations; they are not a claim of universal correctness for an arbitrary execution environment.

## Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Purpose and Status of This Report</b>                   | <b>3</b>  |
| <b>2</b> | <b>Scope, Semantic Profile, and Non-goals</b>              | <b>3</b>  |
| 2.1      | Source configurations . . . . .                            | 3         |
| 2.2      | Admitted OCL validation profile . . . . .                  | 3         |
| 2.3      | Semantic policy . . . . .                                  | 4         |
| 2.4      | Non-goals . . . . .                                        | 5         |
| <b>3</b> | <b>Mathematical Domains and Observations</b>               | <b>6</b>  |
| 3.1      | UML model and object state . . . . .                       | 6         |
| 3.2      | Explicit well-formedness predicates . . . . .              | 7         |
| 3.3      | Property-graph schema and instance . . . . .               | 8         |
| 3.4      | Typed values and environments . . . . .                    | 10        |
| 3.5      | Interpretations . . . . .                                  | 11        |
| <b>4</b> | <b>Complete OCL<sub>val</sub> Language Definition</b>      | <b>11</b> |
| 4.1      | Abstract grammar . . . . .                                 | 11        |
| 4.2      | Type conformance and typing . . . . .                      | 12        |
| 4.3      | Binding and admission derivations . . . . .                | 12        |
| 4.4      | Denotational clauses . . . . .                             | 13        |
| 4.5      | Graph-side denotational clauses . . . . .                  | 16        |
| <b>5</b> | <b>Coordinated Transformation and Refinement Contracts</b> | <b>18</b> |
| 5.1      | Pipeline . . . . .                                         | 18        |
| 5.2      | Concrete schema and instance mappings . . . . .            | 20        |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| 5.3       | Bound-to-semantic constructor map . . . . .         | 21        |
| 5.4       | Relational refinement interface . . . . .           | 21        |
| 5.5       | Normalization rules . . . . .                       | 21        |
| 5.6       | NVA-to-CQM constructor coverage . . . . .           | 22        |
| 5.7       | CQM-to-query-plan realization . . . . .             | 23        |
| 5.8       | Constructor-local realization obligations . . . . . | 23        |
| <b>6</b>  | <b>Graph Adequacy and the Canonical Encoder</b>     | <b>25</b> |
| <b>7</b>  | <b>Preservation Criterion and Assumptions</b>       | <b>26</b> |
| 7.1       | Assumption ledger . . . . .                         | 27        |
| <b>8</b>  | <b>Proof Development</b>                            | <b>29</b> |
| 8.1       | Language-side preservation . . . . .                | 29        |
| 8.2       | Representation-side preservation . . . . .          | 29        |
| 8.3       | Executable realization . . . . .                    | 30        |
| 8.4       | Main theorem . . . . .                              | 30        |
| 8.5       | Dependency and obligation map . . . . .             | 31        |
| <b>9</b>  | <b>Formal Status and Trusted Boundary</b>           | <b>31</b> |
| <b>10</b> | <b>Evaluation Semantics and Evidence</b>            | <b>31</b> |
| <b>11</b> | <b>Evidence Protocol</b>                            | <b>32</b> |
| <b>12</b> | <b>Limitations and Claim Policy</b>                 | <b>33</b> |
| <b>A</b>  | <b>Admitted Surface Summary</b>                     | <b>33</b> |
| <b>B</b>  | <b>NVA-to-CQM Rule Catalog</b>                      | <b>33</b> |

# 1 Purpose and Status of This Report

This report makes the preservation claim independently auditable by fixing the admitted language, semantic domains, coordinated transformations, local proof obligations, main theorem, and evidence boundary. A statement is *normative* when it is a mathematical definition, premise, or result; *realization-specific* when it concerns the selected graph/query interface; and *empirical* when it records a finite observation. These statuses are not interchangeable.

This report accompanies the main paper. All quantitative evidence in Sections 9–11 refers to the clean evidence capture described in the Evidence Protocol. Later experimental runs must be reported separately rather than silently changing the totals fixed here.

Definitions follow the order *Context–Notation–Definition–Where*. A *Where* clause explains newly introduced symbols and adds no hidden premise. The words *preserves* and *correct* refer only to the conditional statements in Sections 7–8; *cross-layer checked* denotes finite agreement, not certification.

The theorem-bearing pipeline is

$$\text{OCL}_{\text{val}} \longrightarrow \text{OVA} \xrightarrow{T_{\text{NORM}}} \text{NVA} \xrightarrow{T_{\text{CQM}}^{\text{spec}}} \text{CQM} \xrightarrow{T_{\text{PLAN}}^{\text{spec}}} \text{QueryPlan} \xrightarrow{\text{emit}} \text{CypherText}.$$

Only the parsed, decoded, bound, semantic, normalized, CQM, and typed query-plan representations in this chain receive theorem-level denotations. Emitted text and parser ASTs are connected to them by explicit refinement relations. Here  $M$  is the finite metamodel and  $I$  the finite object state. Assumptions A1–A9 cover admission/binding, the declared finite-Set and typed-bottom semantics, source and graph well-formedness, graph adequacy, constructor-local query realization, the stable-ID result interface, scalar closure, and the selected Cypher semantics. Their full ledger appears in Section 7; finite evidence never silently discharges them.

## 2 Scope, Semantic Profile, and Non-goals

### 2.1 Source configurations

**Context.** We fix a finite UML model and a finite conforming object state. All later definitions are parameterized by this source configuration unless an explicit graph or query context is stated.

The source of validation is a pair  $(M, I)$  consisting of a finite, well-formed UML domain model and a finite conforming object state. Invariants are evaluated for every instance of their context class, including instances of subclasses. Each source object has an injective, stable identifier. This identifier is the observation preserved across representations; any physical node identifier is never used as a semantic identity. Concrete serialization formats and storage metadata are outside the source denotation and the theorem.

### 2.2 Admitted OCL validation profile

**Notation.** We use  $M$  for a well-formed UML model,  $C$  for an invariant context class, and  $e$  for a surface invariant body. The following definitions first fix the surface carrier and then its admissibility predicate.

**Definition 2.1 (OCLval surface carrier)** Let  $\text{Grammar}_{\text{OCLval}}$  denote the abstract grammar in Section 4. Its constructor carrier is

$$\begin{aligned} \text{Ctor}_{\text{OCLval}} &\triangleq \{\text{constructor tags occurring in } \text{Grammar}_{\text{OCLval}}\}, \\ \text{Surface}_{\text{OCLval}} &\triangleq \text{Constr}(\text{Ctor}_{\text{OCLval}}). \end{aligned}$$

where  $\text{Constr}$  returns the finite constructor trees generated by the given tag set with the arities fixed by the grammar. Let  $\text{Bound}_M$  be the carrier of finite resolved trees returned by the model-aware binder and let  $\text{Ctor}_{\text{Bound}}$  contain all tags that this binder can produce. The admission helpers have signatures

$$\begin{aligned} \text{constructors} &: \text{Bound}_M \rightarrow \mathcal{P}_{\text{fin}}(\text{Ctor}_{\text{Bound}}), \\ \text{captureFree} &: \text{Bound}_M \rightarrow D_{\text{Bool}}, \\ \text{finiteSetOnly} &: \text{Bound}_M \rightarrow D_{\text{Bool}}. \end{aligned}$$

constructors *recursively returns all constructor tags*; captureFree *holds exactly when every variable occurrence is resolved to its intended lexical declaration*; and finiteSetOnly *holds exactly when every reachable collection type is a non-nested finite Set*.

**Where.** Grammar<sub>OCL<sub>val</sub></sub> is the finite abstract grammar in Section 4; Ctor<sub>OCL<sub>val</sub></sub> is its constructor alphabet; and Constr forms finite trees with the arities declared by that grammar. Bound<sub>M</sub> contains model-resolved trees, while Ctor<sub>Bound</sub> is the corresponding bound-tree alphabet. The three helper functions respectively inspect constructor tags, lexical capture, and collection shape. They do not execute an invariant and therefore do not constitute a semantic decision procedure.

**Definition 2.2 (Admission)** *Let  $C$  be a context class and  $e$  an invariant body. The profile OCL<sub>val</sub> admits  $(M, C, e)$  exactly when*

$$\begin{aligned} \text{Admit}_{\text{OCL}_{\text{val}}}(M, C, e) \iff & T_{\text{BIND}}^M(e) = b \text{ succeeds} \wedge \\ & \Gamma_C \vdash_M b : \text{Bool} \wedge \\ & \text{constructors}(b) \subseteq \text{Ctor}_{\text{OCL}_{\text{val}}} \wedge \\ & \text{captureFree}(b) \wedge \text{finiteSetOnly}(b), \end{aligned}$$

where  $\Gamma_C = [\text{self} : \text{Obj}(C)]$ .

**Where.**  $M$  supplies declarations and inheritance,  $C$  is the invariant context, and  $e$  is its surface body.  $b$  is the successfully bound tree;  $\Gamma_C$  contains exactly the distinguished context variable; and Ctor<sub>OCL<sub>val</sub></sub>, captureFree, and finiteSetOnly impose the closed profile. Admission therefore rejects a term when parsing, binding, typing, lexical scope, or profile shape is not established; it does not assign a fallback denotation.

**Definition 2.3 (Admitted invariant domains)** *For a fixed model  $M$  and context class  $C$ , define*

$$\begin{aligned} \text{Inv}_{\text{val}}(M, C) &\triangleq \{e \mid \text{Admit}_{\text{OCL}_{\text{val}}}(M, C, e)\}, \\ \text{Inv}_{\text{val}}(M) &\triangleq \{(C, e) \mid C \in \mathcal{C} \wedge e \in \text{Inv}_{\text{val}}(M, C)\}. \end{aligned}$$

Thus  $\text{Inv}_{\text{val}}(M)$  is a set of context-invariant pairs, whereas  $\text{Inv}_{\text{val}}(M, C)$  is the set of admitted bodies for one context.

**Where.**  $\text{Inv}_{\text{val}}(M, C)$  is the set of admitted bodies for one context class, whereas  $\text{Inv}_{\text{val}}(M)$  is the set of pairs  $(C, e)$  over all classes of  $M$ . The second carrier is therefore not another expression language; it records the context needed by binding and by the later stable-ID observation.

The closed surface contains 47 feature cases, listed in Appendix A. Admission is partial by design. A parsing, binding, typing, capture, or profile failure is a specified rejection, not a counterexample to the preservation theorem.

The normalized type grammar is

$$\tau ::= \beta \mid \text{Set}(\beta), \quad \beta ::= \text{Bool} \mid \text{Int} \mid \text{Real} \mid \text{String} \mid \text{Obj}(C).$$

We write Type for the carrier of these normalized types:

$$\text{Type} \triangleq \{\tau \mid \tau \text{ is generated by the displayed grammar}\}.$$

The type-conformance relation includes reflexivity, conformance of typed bottom, the ordinary UML subclass closure, and the numeric coercions that are explicitly admitted by the binder. Collection conformance is covariant for finite Sets only. A coercion that is not admitted by these binder rules is not silently imported from a broader OCL standard.

## 2.3 Semantic policy

The theorem concerns validation, not arbitrary query equivalence. An invariant is satisfied only when its evaluation is the Boolean value true. Every other reachable Boolean observation, including the declared typed bottom, is validation-false:

$$\text{vtrue}_{\text{OCL}} : D_{\text{Bool}}^\perp \rightarrow D_{\text{Bool}}, \quad \text{vtrue}_G : \widehat{D}_{\text{Bool}}^\perp \rightarrow D_{\text{Bool}},$$

with

$$\text{vtrue}_{\text{OCL}}(v) = \begin{cases} \text{true}, & v = \text{true}, \\ \text{false}, & \text{otherwise.} \end{cases}$$

The graph-side Boolean view  $\text{vtrue}_G$  must commute with value encoding. This policy is explicit because treating a missing value as silently true would change the violation set.

The semantic domains distinguish all of

$$\emptyset, \quad \{\perp_\beta\}, \quad \perp_{\text{Set}(\beta)}.$$

Arrow operations on a scalar or to-one receiver use a finite-Set view:

$$\text{asSet}_{X,\beta}(v) = \begin{cases} \emptyset, & v = \perp_{X,\beta}, \\ \{v\}, & \text{otherwise,} \end{cases} \quad X \in \{D, \widehat{D}\}.$$

This is a deliberately restricted policy, not a claim to reproduce every distinction in the full OMG OCL null/invalid semantics.

The scalar side condition used by the graph and query proofs is

$\text{ScalarClosed}(C, e, M, I) \iff$  for every  $o \in O_C^I$  and every reachable scalar subevaluation  $s$  of  $e$ ,  
integer results remain in `Int64`, real results are finite,  
mixed numeric coercions are exact, divisors are non-zero, and every  
reached operation returns either a profile value or its declared typed bottom.

Equivalently,  $\text{ScalarClosed}$  is a predicate with signature

$$\text{ScalarClosed} : \mathcal{C} \times \text{Surface}_{\text{OCL}_{\text{val}}} \times \text{UMLTuple} \times \text{InstTuple}(M) \rightarrow D_{\text{Bool}}.$$

The same condition is required for each iterator/let environment reached by the evaluation. Thus overflow, non-finite arithmetic, inexact integer-to-real conversion, and division by zero are outside the theorem domain rather than being silently assigned an unspecified result.

Let  $\text{ReachVal}(M, I, G, e, \pi)$  be the finite set of encoded literals, stored scalar values, identifiers, model/association/attribute keys, qualifier payloads, and generated parameter values reachable during this compilation and execution. The bottom-separation premise is

$$\begin{aligned} \text{BottomSeparated}(M, I, G, e, q, \pi) \iff & \text{Params}(q) \subseteq \text{dom}(\pi) \wedge \\ & \perp_G \notin \text{ReachVal}(M, I, G, e, \pi), \end{aligned}$$

with

$$\text{ReachVal} : \text{UMLTuple} \times \text{InstTuple}(M) \times \text{PG}(\Sigma_G) \times \text{Surface}_{\text{OCL}_{\text{val}}} \times \text{QueryPlan} \times \text{ParamEnv}(\Delta) \rightarrow \mathcal{P}_{\text{fin}}(\text{Val}),$$

and

$$\text{BottomSeparated} : \text{UMLTuple} \times \text{InstTuple}(M) \times \text{PG}(\Sigma_G) \times \text{Surface}_{\text{OCL}_{\text{val}}} \times \text{QueryPlan} \times \text{ParamEnv}(\Delta) \rightarrow D_{\text{Bool}}.$$

where  $\perp_G$  denotes the reserved graph representation of typed bottom and  $\text{Params} : \text{QueryPlan} \rightarrow \mathcal{P}_{\text{fin}}(\text{Name})$  returns the declared parameter names. A generated value that collides with  $\perp_G$  is outside the theorem domain; it is never silently reclassified as an ordinary scalar or object identifier.

## 2.4 Non-goals

The theorem excludes Bags, Sequences, OrderedSets, nested collection results, user-defined OCL operations, unrestricted recursion, unconstrained host language calls, and the full OCL null/invalid algebra. It also excludes arbitrary Cypher programs, arbitrary graph encodings, transaction behavior, updates, performance equivalence, and database-internal node identity. The evaluation makes no scalability or statistical-generalization claim.

### 3 Mathematical Domains and Observations

#### 3.1 UML model and object state

**Notation.** The source carriers are `String`, `Name`, `ObjectId`, and the typed value domains  $D_\beta$ ; model-indexed carriers are introduced only after the model tuple is fixed.

Let `String` be the carrier of finite text strings. The lexical and stable-identity carriers are

$$\begin{aligned}\text{Name} &\triangleq \{s \in \text{String} \mid s \text{ is a finite valid lexical identifier}\}, \\ \text{ObjectId} &\triangleq \{s \in \text{String} \mid s \text{ is a finite model-scoped stable identifier}\}.\end{aligned}$$

The primitive value carriers are

$$\begin{aligned}D_{\text{Bool}} &= \{\text{true}, \text{false}\}, & D_{\text{Int}} &= \text{Int64} = \{n \in \mathbb{Z} \mid -2^{63} \leq n \leq 2^{63} - 1\}, \\ D_{\text{Real}} &= \text{Real64} = \{x \mid x \text{ is a finite IEEE-754 binary64 value}\}, & D_{\text{String}} &= \text{String}.\end{aligned}$$

where  $-0.0$  and  $+0.0$  are equal for numeric equality. Object carriers and qualifier carriers are model-indexed below. NaN, infinities, temporal/spatial values, byte arrays, locale collation, and backend-specific scalar values are outside the theorem profile.

**Definition 3.1 (UML domain model)** *A UML domain model is the tuple*

$$M = (\mathcal{C}, \mathcal{A}, \mathcal{R}, \preceq, \text{owner}, \text{type}_A, \text{src}, \text{tgt}, \text{opp}, \text{mult}, \text{qual}) \in \text{UML}_{WF}.$$

$\mathcal{C}$ ,  $\mathcal{A}$ , and  $\mathcal{R}$  are finite sets of classes, attributes, and directed roles. The remaining maps record inheritance, attribute ownership and types, role endpoints and opposites, multiplicities, and ordered qualifier declarations.  $\text{UML}_{WF}$  requires unique names in their declared scopes, well-founded inheritance, well-typed ownership, compatible opposites, and well-formed multiplicities and qualifiers.

**Where.**  $\mathcal{C}$ ,  $\mathcal{A}$ , and  $\mathcal{R}$  are the finite class, attribute, and directed-role carriers.  $\preceq$  is the subclass relation; `owner` and  $\text{type}_A$  give declaration ownership and attribute type; `src`, `tgt`, and `opp` give role endpoints and opposites; `mult` gives lower/upper multiplicities; and `qual` gives the ordered qualifier declarations. The predicate  $\text{UML}_{WF}$  is the set of tuples satisfying the well-formedness clauses in Definition 3.4.

Formally, the carrier notation is closed by

$$\text{UML}_{WF} = \{M \mid \text{UMLWF}(M)\}, \quad \text{Inst}_{WF}(M) = \{I \mid \text{InstWF}(M, I)\}.$$

Thus membership in either carrier is predicate-backed set membership, not an uninterpreted validity flag.

**Definition 3.2 (Model-indexed qualifier carrier)** *For a role  $r$  whose ordered qualifier declarations are  $(q_{r,1} : \beta_{r,1}, \dots, q_{r,k_r} : \beta_{r,k_r})$ , define*

$$\text{Qual}_M(r) \triangleq \prod_{i=1}^{k_r} D_{\beta_{r,i}}^\perp, \quad \mathbf{Q}_M(r) \triangleq \text{Qual}_M(r), \quad \mathbf{Q}_M^* \triangleq \bigsqcup_{r \in \mathcal{R}} (\{r\} \times \text{Qual}_M(r)).$$

*The empty product is the singleton empty tuple, written  $\langle \rangle$ . The role component determines which qualifier signature applies, so tuples belonging to different roles are not implicitly identified.*

Let  $\text{Value}^\perp \triangleq \bigsqcup_{\tau \in \text{Type}} D_\tau^\perp$ ; the disjoint union retains the declared type tag, including the tag of each typed bottom.

**Where.**  $q_{r,i}$  is the  $i$ th qualifier name of role  $r$ ,  $\beta_{r,i}$  is its declared primitive type, and  $k_r$  is the qualifier arity.  $D_\beta^\perp$  is the ordinary carrier of values of type  $\beta$  plus its typed bottom;  $\mathbf{Q}_M(r) = \text{Qual}_M(r)$  is the role-specific product, and  $\mathbf{Q}_M^*$  is a disjoint union that retains the role tag.

**Definition 3.3 (Object state)** *A conforming object state is*

$$I = (O, \text{classOf}_I, \text{val}_I, \text{link}_I, \text{id}_I) \in \text{Inst}_{WF}(M).$$

*$O$  is finite;  $\text{classOf}_I$  gives the most specific class; with  $\text{AppAttr}_I = \{(o, a) \mid \text{classOf}_I(o) \preceq \text{owner}(a)\}$ ,  $\text{val}_I : \text{AppAttr}_I \rightarrow \text{Value}^\perp$  is total and  $\text{val}_I(o, a) \in D_{\text{type}_A(a)}^\perp$ ;  $\text{link}_I \subseteq \{(o, r, o', \bar{q}) \mid o, o' \in O, r \in \mathcal{R}, \bar{q} \in \text{Qual}_M(r)\}$  records directed, optionally qualified links; and  $\text{id}_I : O \rightarrow \text{ObjectId}$  is injective. For class  $C$ ,*

$$O_C^I = \{o \in O \mid \text{classOf}_I(o) \preceq C\}.$$

**Where.**  $O$  is the finite object carrier;  $\text{classOf}_I$  returns each object's most-specific class;  $\text{val}_I$  is typed attribute lookup;  $\text{link}_I$  is the set of directed role links with their role-indexed qualifier tuples; and  $\text{id}_I$  is the injective, model-scoped stable-ID map.  $O_C^I$  is the class extent including subclasses because  $o \in O_C^I$  exactly when  $\text{classOf}_I(o) \preceq C$ . Pairs outside  $\text{AppAttr}_I$  are non-applicable; for an applicable pair,  $\perp_\tau$  is the unique absent-value observation. Partial undefinedness and typed bottom are therefore not competing meanings of missing data.

The source observers needed by the profile are stable identity, dynamic type, attribute lookup, directed navigation (including reverse and qualified navigation), and class extent. No additional source-state observation may be introduced in a proof case without extending the adequacy contract.

### 3.2 Explicit well-formedness predicates

The abbreviations  $\text{UML}_{WF}$  and  $\text{Inst}_{WF}(M)$  are defined by the following clauses; they are not uninterpreted quotation marks around valid model flags. The underlying tuple carriers are

$\text{UMLTuple} \triangleq \{(\mathcal{C}, \mathcal{A}, \mathcal{R}, \preceq, \text{owner}, \text{type}_A, \text{src}, \text{tgt}, \text{opp}, \text{mult}, \text{qual}) \mid \text{all components are finite carriers/maps of the declared domains}\}$   
 $\text{InstTuple}(M) \triangleq \{(O, \text{classOf}_I, \text{val}_I, \text{link}_I, \text{id}_I) \mid O \text{ and } \text{link}_I \text{ are finite}\}.$

The well-formedness predicates have the signatures

$$\text{UMLWF} : \text{UMLTuple} \rightarrow D_{\text{Bool}}, \quad \text{InstWF} : \text{UMLTuple} \times \text{InstTuple}(M) \rightarrow D_{\text{Bool}}.$$

#### Definition 3.4 (UML well-formedness)

$\text{UMLWF}(M) \iff \mathcal{C}, \mathcal{A}, \mathcal{R} \text{ are finite}$   
 $\wedge \preceq \text{ is a reflexive, transitive, antisymmetric relation on } \mathcal{C}$   
 $\wedge \text{owner} : \mathcal{A} \cup \mathcal{R} \rightarrow \mathcal{C}$   
 $\wedge \text{type}_A(a) \text{ is defined for every } a \in \mathcal{A}$   
 $\wedge \text{src}(r), \text{tgt}(r) \in \mathcal{C} \text{ for every } r \in \mathcal{R}$   
 $\wedge \text{opp}(r) = r' \Rightarrow \text{opp}(r') = r \text{ with reversed endpoints}$   
 $\wedge \text{mult}(r) = [\ell_r, u_r] \text{ with } 0 \leq \ell_r \leq u_r \leq \infty$   
 $\wedge \text{qual}(r) \text{ is an ordered list of distinct, well-typed declarations}$   
 $\wedge \text{all names are unique in their model scopes}$   
 $\wedge \text{role and attribute owners agree with their declared endpoints}.$

*The inheritance relation is acyclic, every referenced class and type exists, and every opposite or qualifier is resolved in  $M$ .*

**Where.** The displayed conjunction is a predicate on the tuple  $M$ , not a Boolean field stored in a model file. Finiteness makes extents and navigation results finite; relation and map clauses make ownership, typing, endpoints, opposites, multiplicities, names, and qualifier products total on their declared domains.

#### Definition 3.5 (Instance well-formedness) For $M$ satisfying Definition 3.4,

$\text{InstWF}(M, I) \iff O \text{ and } \text{link}_I \text{ are finite and } \text{classOf}_I : O \rightarrow \mathcal{C}$   
 $\wedge \text{classOf}_I(o) \text{ is the most-specific declared class of } o$   
 $\wedge \text{val}_I : \text{AppAttr}_I \rightarrow \text{Value}^\perp \text{ is total and correctly typed on every } (o, a) \in \text{AppAttr}_I$   
 $\wedge \text{id}_I : O \hookrightarrow \text{ObjectId}$   
 $\wedge \text{link}_I \subseteq \{(o, r, o', \bar{q}) \mid o, o' \in O, r \in \mathcal{R}, \bar{q} \in \text{Qual}_M(r)\}$   
 $\wedge \text{links have role-compatible endpoints and opposite pairs}$   
 $\wedge \text{multiplicities hold and qualifier tuples have declared arity/types}$   
 $\wedge \text{qualified links are unique under role/key and no reference dangles}.$

*The notation  $I \in \text{Inst}_{WF}(M)$  means  $\text{InstWF}(M, I)$ .*

**Where.** The predicate quantifies over every applicable attribute and link in  $I$ . Finiteness of  $\text{link}_I$  is explicit because it is used to prove finiteness of the canonical edge carrier. “Applicable” means that the attribute owner conforms to  $\text{classOf}_I(o)$ ; an applicable absent observation is represented by the declared typed bottom, while a non-applicable pair lies outside  $\text{AppAttr}_I$ . The endpoint, multiplicity, opposite, qualifier, identity, and non-dangling clauses are exactly the source obligations used by navigation, all-instances, and stable-ID preservation; no persistence-layer convention is implicit.

These predicates explain why A2 is a domain premise rather than a test label: all source observations used by the proof are total and typed only on these domains.

### 3.3 Property-graph schema and instance

**Context.** Fix a target schema  $\Sigma_G$  obtained from the source model and a graph instance  $G$  over that schema. The next definitions distinguish the abstract carrier from the well-formedness predicate and from concrete observers. The graph-value carrier used by property and qualifier maps is

$$\text{Val} \triangleq \text{ObjectId} \uplus \bigcup_{\tau} \widehat{D}_{\tau}^{\perp},$$

with the type tag retained by each summand. The graph storage-type carrier and role-tagged qualifier carriers are

$$\begin{aligned} \text{GType} &\triangleq \{\text{Bool}, \text{Int}, \text{Real}, \text{String}, \text{ObjectId}\} \cup \{\text{List}(g) \mid g \in \text{GType}\}, \\ \widehat{\text{Qual}}_{\Sigma_G}(R_r) &\triangleq \{R_r\} \times \prod_{i=1}^{k_r} \widehat{D}_{\beta_{r,i}}^{\perp}, \\ \widehat{\text{Qual}}_{\Sigma_G} &\triangleq \bigsqcup_{R_r \in \mathcal{R}_T} \widehat{\text{Qual}}_{\Sigma_G}(R_r). \end{aligned}$$

The product is empty (and therefore a singleton) for an unqualified role; the leading role tag prevents qualifier tuples of different relationship types from being identified. For the layered encoding, write

$$G = G_{M2} \cup G_{M1} \cup G_{\text{typing}},$$

where  $G_{M2}$  exposes metamodel declarations,  $G_{M1}$  exposes object and link instances, and  $G_{\text{typing}}$  exposes conformance projections. The resolver view recovered from  $G_{M2}$  must be isomorphic to the source model resolver for every reachable class, attribute, role, inheritance, and qualifier lookup. This is a schema-recovery obligation, not inference from untyped instance links.

**Definition 3.6 (Property-graph representation)** *A target schema and graph have the signatures*

$$\begin{aligned} \Sigma_G &= (\mathcal{N}_T, \mathcal{R}_T, \mathcal{P}_D, \mathcal{K}_D, \text{src}_T, \text{tgt}_T, \text{owner}_P, \text{type}_P, \text{meta}_T) \in \text{GraphSchema}, \\ G &= (V, E, \text{src}_G, \text{tgt}_G, \text{label}_V, \text{label}_E, \text{prop}, \text{qualifier}) \in \text{PG}(\Sigma_G). \end{aligned}$$

*The schema records node and relationship types, properties, keys, endpoints, ownership, value types, object-label metadata, and qualifier declarations. The finite graph has total edge endpoints, typed labels, a partial typed property map over nodes and edges, and one encoded qualifier tuple per edge. The canonical stable-ID property is  $p_{id}^M = \text{useId} : \text{ObjectId}$ .*

**Where.**  $\Sigma_G$  is the target schema and  $G$  is an instance of that schema.  $\mathcal{N}_T$  and  $\mathcal{R}_T$  are node and relationship types;  $\mathcal{P}_D$  and  $\mathcal{K}_D$  are property and key declarations;  $\text{src}_T, \text{tgt}_T$  are relationship endpoints;  $\text{owner}_P, \text{type}_P$ , and  $\text{meta}_T$  record property ownership, storage type, and model-label metadata. In  $G$ ,  $V$  and  $E$  are disjoint finite node and edge carriers, the two endpoint maps are total,  $\text{prop}$  stores typed values on either carrier, and  $\text{qualifier}$  stores the role-indexed encoded qualifier. The property  $\text{useId}$  is the only identity observation used by the theorem. For the selected canonical profile, relationship metadata also contains an injective association key, source and target role keys, direction, model scope, and ordered qualifier payloads for both navigation directions. R5–R6 use these fields to exclude a relationship belonging to another association or role; an encoding that omits one of them must discharge an equivalent adequacy obligation before it is admitted.

**Definition 3.7 (Property-graph carrier domains)** *The schema carrier is*

$$\text{GraphSchema} \triangleq \{\Sigma_G = (\mathcal{N}_T, \mathcal{R}_T, \mathcal{P}_D, \mathcal{K}_D, \text{src}_T, \text{tgt}_T, \text{owner}_P, \text{type}_P, \text{meta}_T)\}.$$

*For a fixed schema  $\Sigma_G$ , the graph carrier is*

$$\text{PG}(\Sigma_G) \triangleq \left\{ G = (V, E, \text{src}_G, \text{tgt}_G, \text{label}_V, \text{label}_E, \text{prop}, \text{qualifier}) \mid \begin{array}{l} V, E \text{ are finite, } \text{src}_G, \text{tgt}_G : E \rightarrow V, \\ \text{label}_V : V \rightarrow \mathcal{N}_T, \text{label}_E : E \rightarrow \mathcal{R}_T, \\ \text{prop} : (V \cup E) \times \mathcal{P}_D \rightarrow \text{Val}, \\ \text{qualifier} : E \rightarrow \widehat{\text{Qual}}_{\Sigma_G} \text{ is total} \end{array} \right\}.$$

*SchemaWF and GraphWF select the well-formed members of these carriers; they are not additional informal flags.*

**Definition 3.8 (Schema and graph well-formedness)** *The predicates used throughout the report have the signatures*

$$\text{SchemaWF} : \text{GraphSchema} \rightarrow D_{\text{Bool}}, \quad \text{GraphWF} : \text{GraphSchema} \times \text{PG}(\Sigma_G) \rightarrow D_{\text{Bool}}.$$

*For  $\Sigma_G = (\mathcal{N}_T, \mathcal{R}_T, \mathcal{P}_D, \mathcal{K}_D, \dots)$ ,*

$$\begin{aligned} \text{SchemaWF}(\Sigma_G) &\iff \mathcal{N}_T, \mathcal{R}_T, \mathcal{P}_D, \mathcal{K}_D \text{ are finite} \wedge \text{src}_T, \text{tgt}_T \text{ are total on } \mathcal{R}_T \\ &\quad \wedge \text{owner}_P, \text{type}_P \text{ are total on declared properties} \wedge \text{all keys, labels, endpoints, and qualifier declarations} \end{aligned}$$

$$\begin{aligned} \text{GraphWF}(\Sigma_G, G) &\iff G \in \text{PG}(\Sigma_G) \wedge V, E \text{ are finite and disjoint} \wedge \text{src}_G, \text{tgt}_G \text{ are total} \wedge \\ &\quad \text{label}_V, \text{label}_E, \text{prop}, \text{qualifier} \text{ use only declared labels, owners, types, keys, and qualifier signatures} \wedge \\ &\quad \text{edge endpoints conform to } \Sigma_G; \text{ every validation-visible object has} \\ &\quad \text{one model-scoped identity property; no dangling or out-of-scope fact occurs.} \end{aligned}$$

**Where.** *SchemaWF* checks the target declaration layer; *GraphWF* checks one finite instance against that declaration. The final clauses include object-label metadata, edge/qualifier ownership, typed properties, identity uniqueness, endpoint conformance, and scope closure, which are precisely the well-formedness facts consumed by R1–R7 and the canonical lemmas.

**Where.** *GType* is the storage-type carrier; *Val* is the disjoint union of stable identifiers and encoded typed values; and  $\widehat{\text{Qual}}_{\Sigma_G}$  is the role-tagged encoded qualifier carrier.  $\text{PG}(\Sigma_G)$  contains exactly finite tuples with disjoint nodes and edges, declared labels, total endpoints, a partial property map on nodes or edges, and a total role-compatible qualifier map. The predicates *SchemaWF* and *GraphWF* below select the members that may be used by the theorem.

**Definition 3.9 (Concrete graph observers)** *Abstract observers isolate the proof from a concrete label spelling:*

$$\text{InstanceOf}_G(v, C), \quad \text{read}_G^a(v), \quad \text{Link}_G(v, r, v', Q), \quad \text{allInstances}_G(C).$$

*The relevant object-node extent is*

$$V_{\text{obj}}(G) = \{v \in V \mid \text{label}_V(v) \in \text{ObjLabels}(\Sigma_G)\}, \quad V_C(G) = \{v \in V_{\text{obj}}(G) \mid \text{InstanceOf}_G(v, C)\}.$$

*For declared  $a : \text{Obj}(C') \rightarrow \tau$  and role  $r : C_s \rightarrow C_t$ , the observers used by the denotational clauses have the signatures*

$$\begin{aligned} \text{InstanceOf}_G &: \widehat{D}_{\text{Obj}(C')}^\perp \times \mathcal{C} \rightarrow D_{\text{Bool}}^\perp, \\ \text{read}_G^a &: \widehat{D}_{\text{Obj}(C')}^\perp \rightarrow \widehat{D}_\tau^\perp, \\ \text{Link}_G &: \widehat{D}_{\text{Obj}(C_s)}^\perp \times \{r\} \times \widehat{D}_{\text{Obj}(C_t)}^\perp \times \widehat{\text{Qual}}_{\Sigma_G}(R_r) \rightarrow D_{\text{Bool}}, \\ \text{allInstances}_G &: \mathcal{C} \rightarrow \mathcal{P}_{\text{fin}}(\widehat{D}_{\text{Obj}(C)}), \\ \text{nav}_{\text{graph}} &: \widehat{D}_{\text{Obj}(C_s)}^\perp \times \{r\} \times \widehat{\text{Qual}}_{\Sigma_G}(R_r) \rightarrow \widehat{D}_{\tau_r}^\perp. \end{aligned}$$

*The concrete read observer is total on its typed carrier:*

$$\text{read}_G^a(v) = \begin{cases} \text{prop}(v, p_a), & v \in V_{\text{obj}}(G) \text{ and } a \text{ is applicable to } v, \\ \perp_\tau, & \text{otherwise.} \end{cases}$$

**Where.**  $\text{InstanceOf}_G$ ,  $\text{read}_G^a$ ,  $\text{Link}_G$ , and  $\text{allInstances}_G$  are abstract typed observers over a well-formed graph.  $V_{\text{obj}}(G)$  contains exactly nodes carrying schema-declared object labels, and  $V_C(G)$  is the corresponding extent for class  $C$ . A missing or inapplicable property is returned as the declared typed graph bottom; it is not confused with an empty Set or a missing row.

### 3.4 Typed values and environments

**Notation.** For each normalized type  $\tau$ , a hat distinguishes its graph-side carrier from the corresponding source-side carrier. Typed bottoms are indexed by the same type so that they cannot be confused with ordinary values or empty Sets.

For every normalized type  $\tau$ , let

$$D_\tau^\perp = D_\tau \cup \{\perp_\tau\}, \quad \widehat{D}_\tau^\perp = \widehat{D}_\tau \cup \{\widehat{\perp}_\tau\}.$$

In particular,

$$\begin{aligned} D_{\text{Obj}(C)} &= O_C^I, \\ D_{\text{Set}(\beta)}^\perp &= \mathcal{P}_{\text{fin}}(D_\beta^\perp) \cup \{\perp_{\text{Set}(\beta)}\}, \\ \widehat{D}_{\text{Obj}(C)} &= V_C(G), \\ \widehat{D}_{\text{Set}(\beta)}^\perp &= \mathcal{P}_{\text{fin}}(\widehat{D}_\beta^\perp) \cup \{\widehat{\perp}_{\text{Set}(\beta)}\}. \end{aligned}$$

Let

$$\text{TypeEnv} \triangleq \{\Gamma \mid \Gamma : \text{Name} \rightarrow \text{Type}, \text{dom}(\Gamma) \text{ is finite}\}.$$

Take  $\Gamma \in \text{TypeEnv}$ , where  $\text{Type}$  is the normalized type grammar of Section 2. The source and graph environment carriers are

$$\begin{aligned} \text{Env}_I(\Gamma) &\triangleq \{\rho \mid \text{dom}(\rho) = \text{dom}(\Gamma), \forall x \in \text{dom}(\Gamma), \rho(x) \in D_{\Gamma(x)}^\perp\}, \\ \text{Env}_G(\Gamma) &\triangleq \{\gamma \mid \text{dom}(\gamma) = \text{dom}(\Gamma), \forall x \in \text{dom}(\Gamma), \gamma(x) \in \widehat{D}_{\Gamma(x)}^\perp\}. \end{aligned}$$

Thus source and graph environments are type-respecting maps

$$\rho \in \text{Env}_I(\Gamma), \quad \gamma \in \text{Env}_G(\Gamma).$$

Given an object-node map  $\eta$ , write  $\rho \sim_\eta \gamma$  when primitive values are related by their encoder, object values by  $\eta$ , finite Sets pointwise, and each typed bottom by its declared separated graph representation. Formally, with equal domains  $\text{dom}(\rho) = \text{dom}(\gamma) = \text{dom}(\Gamma)$ ,

$$\rho \sim_\eta \gamma \iff \forall x \in \text{dom}(\Gamma) : \begin{cases} \gamma(x) = \eta(o), & \Gamma(x) = \text{Obj}(C), \rho(x) = o, \\ \gamma(x) = \text{encodeValue}_{\Gamma(x)}(v), & \rho(x) = v \neq \perp_{\Gamma(x)}, \\ \gamma(x) = \widehat{\perp}_{\Gamma(x)}, & \rho(x) = \perp_{\Gamma(x)}. \end{cases}$$

For a finite-Set type, the second clause applies the encoder pointwise; for an object set, its element encoder is  $\eta$ .

The value encoder has type

$$\text{encodeValue}_\tau : D_\tau^\perp \rightarrow \widehat{D}_\tau^\perp$$

and satisfies

$$\begin{aligned} \text{encodeValue}_{\text{Obj}(C)}(o) &= \eta(o), \\ \text{encodeValue}_\tau(\perp_\tau) &= \widehat{\perp}_\tau, \\ \text{encodeValue}_{\text{Set}(\beta)}(S) &= \{\text{encodeValue}_\beta(x) \mid x \in S\}, \\ \text{encodeValue}_{\text{Set}(\beta)} \circ \text{asSet}_{D,\beta} &= \text{asSet}_{\widehat{D},\beta} \circ \text{encodeValue}_\beta, \\ \text{vtrue}_G(\text{encodeValue}_{\text{Bool}}(v)) &= \text{vtrue}_{\text{OCL}}(v). \end{aligned}$$

For every role  $r$ , the qualifier encoder is the typed product map

$$\begin{aligned} \text{encodeQual}_{M,\Sigma_G,r} : \text{Qual}_M(r) &\rightarrow \widehat{\text{Qual}}_{\Sigma_G}(R_r), \\ \text{encodeQual}_{M,\Sigma_G,r}(\langle q_1, \dots, q_{k_r} \rangle) &= \langle \text{encodeValue}_{\beta_{r,1}}(q_1), \dots, \\ &\quad \text{encodeValue}_{\beta_{r,k_r}}(q_{k_r}) \rangle. \end{aligned}$$

We abbreviate this value by  $Q_{r,\bar{q}}^M \triangleq \text{encodeQual}_{M,\Sigma_G,r}(\bar{q})$ ; the qualifier codec is injective on each role-indexed carrier.

**Where.**  $D_\tau^\perp$  and  $\widehat{D}_\tau^\perp$  are source and graph carriers with distinct typed bottoms. Object carriers are the source and graph class extents; finite Set carriers contain finite sets of element values plus a separate collection bottom. **TypeEnv** is a finite partial map from names to normalized types, while  $\text{Env}_I(\Gamma)$  and  $\text{Env}_G(\Gamma)$  are the type-respecting source and graph bindings. The relation  $\rho \sim_\eta \gamma$  means that primitive values, objects, Sets, and typed bottoms correspond through the encoder and object-node map  $\eta$ .

### 3.5 Interpretations

The expression carriers used by the interpretations are

$$\begin{aligned} \text{OCLExpr}_\tau(M, \Gamma) &\triangleq \{e \in \text{Surface}_{\text{OCL}_{\text{val}}} \mid \Gamma \vdash_M e : \tau\}, \\ \text{SrcExpr}_\tau(M, \Gamma) &\triangleq \{u \mid u \text{ is a finite resolved source-semantic tree of type } \tau \text{ under } M, \Gamma\}, \\ \text{NormExpr}_\tau(M, \Gamma) &\triangleq \{n \in \text{SrcExpr}_\tau(M, \Gamma) \mid n \text{ contains no derived operator}\}. \end{aligned}$$

**Where.**  $\text{OCLExpr}_\tau$  contains admitted surface trees,  $\text{SrcExpr}_\tau$  contains resolved source trees, and  $\text{NormExpr}_\tau$  excludes derived operators. The three brackets are total only on their displayed typed carriers: source evaluation returns  $D_\tau^\perp$ , while graph evaluation returns its encoded counterpart  $\widehat{D}_\tau^\perp$ . The environment argument is therefore part of the semantic interface, not an implicit global state. Section 5 refines **SrcExpr** into the bound, OVA, and NVA subcarriers and identifies **NormExpr** with the admitted NVA normal forms.

The proof uses three typed interpretations:

$$\begin{aligned} \llbracket \cdot \rrbracket_{\text{OCL}}^{M, I} &: \text{OCLExpr}_\tau(M, \Gamma) \times \text{Env}_I(\Gamma) \rightarrow D_\tau^\perp, \\ \llbracket \cdot \rrbracket_{\text{src}}^{M, I} &: \text{SrcExpr}_\tau(M, \Gamma) \times \text{Env}_I(\Gamma) \rightarrow D_\tau^\perp, \\ \llbracket \cdot \rrbracket_{\text{graph}}^{\Sigma_G, G} &: \text{NormExpr}_\tau(M, \Gamma) \times \text{Env}_G(\Gamma) \rightarrow \widehat{D}_\tau^\perp. \end{aligned}$$

The first gives the admitted surface meaning, the second gives bound, semantic, and normalized source meaning, and the third evaluates normalized terms through graph observations. Keeping these interpretations distinct prevents parser, representation, and backend agreement from being hidden in a single black-box predicate.

## 4 Complete $\text{OCL}_{\text{val}}$ Language Definition

### 4.1 Abstract grammar

Let  $x$  range over variables,  $\ell$  over typed literals,  $a$  over resolved attributes,  $r$  over resolved directed roles,  $C$  over classes,  $\bar{q}$  over qualifier tuples, and  $\odot$  over the declared scalar operators. The admitted abstract grammar is

$$\begin{aligned} e ::= & x \mid \ell \mid e.a \mid e.r[\bar{q}] \mid C.\text{allInstances}() \\ & \mid \neg e \mid e_1 \odot e_2 \mid \text{if } e_c \text{ then } e_t \text{ else } e_f \\ & \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{Set}\{e_1, \dots, e_k\} \\ & \mid e \rightarrow k(x \mid p) \mid e \rightarrow \text{collect}(x \mid b) \\ & \mid e \rightarrow m(e') \mid e \rightarrow c(e') \mid e \rightarrow \text{size}() \mid e \rightarrow \text{isEmpty}() \\ & \mid e \rightarrow \text{notEmpty}() \mid e \rightarrow \text{asSet}() \mid e.\text{oclIsKindOf}(C) \mid e.\text{oclAsType}(C), \end{aligned}$$

where

$$\begin{aligned} k &\in \{\text{exists}, \text{forAll}, \text{select}, \text{reject}, \text{isUnique}\}, \\ m &\in \{\text{includes}, \text{excludes}, \text{includesAll}, \text{excludesAll}\}, \\ c &\in \{\text{union}, \text{intersection}\}. \end{aligned}$$

The four nullary Set observations are explicit productions; they are not obtained by assigning an implicit arity to  $m$  or  $c$ . Unary Boolean negation is the explicit  $\neg e$  production. The binary scalar-operator family  $\odot$  comprises Boolean  $\wedge, \vee, \text{xor}, \Rightarrow$ , equality/inequality, order, and  $+, -, \times, /$  with the typing rules below. Surface spelling is accepted only when it binds to exactly one constructor in this grammar.

## 4.2 Type conformance and typing

Write  $\sigma \leq_M \tau$  for admissible type conformance. Its generating rules are reflexivity, typed-bottom conformance, direct/transitive UML subclassing,  $\text{Int} \leq \text{Real}$ , and finite-Set covariance. The admitted profile does not infer unlisted transitive numeric coercions. The central judgment is  $\Gamma \vdash_M e : \tau$ .

Table 1: Typing-rule schemas for  $\text{OCL}_{\text{val}}$ .

| Rule family | Premises and conclusion                                                                                                                                                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T-VAR       | $x : \tau \in \Gamma \Rightarrow \Gamma \vdash_M x : \tau$                                                                                                                                                                        |
| T-LIT       | $\ell \in D_\beta^\perp \Rightarrow \Gamma \vdash_M \ell : \beta$                                                                                                                                                                 |
| T-ATTR      | $\Gamma \vdash_M e : \text{Obj}(C'), C' \preceq C, \text{owner}(a) = C \Rightarrow \Gamma \vdash_M e.a : \text{type}_A(a)$                                                                                                        |
| T-NAV       | Receiver conforms to $\text{src}(r)$ and qualifier terms match $\text{qual}(r)$ ; the result is $\text{Obj}(\text{tgt}(r))$ for upper multiplicity one and $\text{Set}(\text{Obj}(\text{tgt}(r)))$ otherwise                      |
| T-ALL       | $C \in \mathcal{C} \Rightarrow \Gamma \vdash_M C.\text{allInstances}() : \text{Set}(\text{Obj}(C))$                                                                                                                               |
| T-BOOL      | Boolean operands yield Boolean for not/and/or/xor/implies                                                                                                                                                                         |
| T-COMP      | Operands have a defined common coercion target; equality yields Boolean, order is restricted to ordered scalar types                                                                                                              |
| T-ARITH     | Numeric operands have a defined numeric join; result is that join except division, whose declared result is Real                                                                                                                  |
| T-IF        | Condition is Boolean and branch types have a least admitted common supertype $\tau$ ; result type is $\tau$                                                                                                                       |
| T-LET       | $\Gamma \vdash_M e_1 : \tau_1$ and $\Gamma, x : \tau_1 \vdash_M e_2 : \tau_2$ with fresh $x$ ; result is $\tau_2$                                                                                                                 |
| T-ITER      | Receiver has finite-Set view $\text{Set}(\beta)$ and the body is typed under $\Gamma, x : \beta$ ; predicate iterators require Boolean bodies, collect requires a scalar body, and isUnique requires a scalar projection body     |
| T-COLLECT   | A Set receiver and scalar body $\tau$ yield $\text{Set}(\tau)$ ; nested collection results are rejected                                                                                                                           |
| T-SET       | Elements have a defined common scalar type $\beta$ ; result is $\text{Set}(\beta)$                                                                                                                                                |
| T-SETOP     | Set relations require compatible element types; union/intersection return the common Set type; size returns Int; isEmpty, notEmpty, and isUnique return Bool; and asSet returns $\text{Set}(\beta)$ for the declared element type |
| T-TYPE      | oclIsKindOf returns Bool; an admitted cast returns $\text{Obj}(C)$ and is defined only for model-related classes                                                                                                                  |

The collection operators have the following explicit typing instances (with  $\beta$  fixed by the receiver and  $\tau$  by the body):

$$\begin{array}{c}
\frac{\Gamma \vdash_M S : \text{Set}(\beta) \quad \Gamma, x : \beta \vdash_M p : \text{Bool}}{\Gamma \vdash_M S \rightarrow \text{exists}(x \mid p) : \text{Bool}}, \quad \frac{\Gamma \vdash_M S : \text{Set}(\beta) \quad \Gamma, x : \beta \vdash_M b : \tau \quad \tau \text{ scalar}}{\Gamma \vdash_M S \rightarrow \text{collect}(x \mid b) : \text{Set}(\tau)}, \\
\frac{\Gamma \vdash_M S : \text{Set}(\beta) \quad \Gamma, x : \beta \vdash_M b : \tau \quad \tau \text{ scalar}}{\Gamma \vdash_M S \rightarrow \text{isUnique}(x \mid b) : \text{Bool}}, \quad \frac{\Gamma \vdash_M S : \text{Set}(\beta)}{\Gamma \vdash_M S \rightarrow \text{notEmpty}() : \text{Bool}}, \\
\frac{\Gamma \vdash_M v : \text{Obj}(C) \text{ or scalar } \beta}{\Gamma \vdash_M v \rightarrow \text{asSet}() : \text{Set}(\beta)}.
\end{array}$$

The forAll, select, and reject rules use the same receiver/body premises as exists; union, intersection, membership, and cardinality require compatible finite-Set element types and return the types listed in Table 1.

## 4.3 Binding and admission derivations

Binding is the partial, model-indexed judgment

$$M; \Gamma \vdash e \Downarrow b : \tau.$$

It resolves a surface name to one declaration and records its stable model identity, declared type, direction, multiplicity, qualifier signature, and lexical symbol identifier. The core rules are:

$$\frac{x : \tau \in \Gamma}{M; \Gamma \vdash x \Downarrow \text{Var}(x, s_x, \tau) : \tau} \quad \frac{M; \Gamma \vdash e \Downarrow b : \text{Obj}(C') \quad C' \preceq C \quad \text{owner}(a) = C}{M; \Gamma \vdash e.a \Downarrow \text{Attr}(b, a, C, \text{type}_A(a)) : \text{type}_A(a)}.$$

Navigation resolves exactly one role in the requested direction and checks the qualifier tuple. Let and iterator rules extend  $\Gamma$  with one fresh symbol; alpha-renaming is performed before extension when a

capture would otherwise occur. A derivation is rejected on ambiguity, unresolved names, undefined coercion joins, illegal collection shape, unbound variables, or a constructor outside the grammar. Thus

$$\text{Admit}_{\text{OCL}_{\text{val}}}(M, C, e) \iff \exists b. M; [\text{self} : \text{Obj}(C)] \vdash e \Downarrow b : \text{Bool} \wedge \text{captureFree}(b) \wedge \text{finiteSetOnly}(b).$$

#### 4.4 Denotational clauses

For every admitted primitive operator  $f$  of result type  $v$ , define the typed strict lift

$$\text{strict}_{k,f} : D_{\tau_1}^\perp \times \cdots \times D_{\tau_k}^\perp \rightarrow D_v^\perp$$

for  $k \geq 1$ , where the declared primitive  $f$  has ordinary signature  $D_{\tau_1} \times \cdots \times D_{\tau_k} \rightarrow D_v$ , and define it by returning  $\perp_v$  if any argument is typed bottom or if  $f$  is undefined on the ordinary arguments, and otherwise applying  $f$ . We write  $\text{strict}_k(f)$  when the result type is fixed by the typing derivation. Thus division by zero, for example, yields the declared numeric bottom. Primitive arithmetic, comparison, and equality use the corresponding typed strict lift after the typing derivation fixes a common coercion target. Boolean operators use the following complete three-valued table, where  $\mathbf{t} = \text{true}$ ,  $\mathbf{f} = \text{false}$ , and  $\mathbf{u} = \perp_{\text{Bool}}$ . For each displayed binary operator  $\odot$ ,

$$\text{BoolOp}_\odot : D_{\text{Bool}}^\perp \times D_{\text{Bool}}^\perp \rightarrow D_{\text{Bool}}^\perp$$

is the total function defined by the table:

| $a$          | $b$          | $a \wedge b$ | $a \vee b$   | $a \Rightarrow b$ | $a \text{ xor } b$ |
|--------------|--------------|--------------|--------------|-------------------|--------------------|
| $\mathbf{t}$ | $\mathbf{t}$ | $\mathbf{t}$ | $\mathbf{t}$ | $\mathbf{t}$      | $\mathbf{f}$       |
| $\mathbf{t}$ | $\mathbf{f}$ | $\mathbf{f}$ | $\mathbf{t}$ | $\mathbf{f}$      | $\mathbf{t}$       |
| $\mathbf{t}$ | $\mathbf{u}$ | $\mathbf{u}$ | $\mathbf{t}$ | $\mathbf{u}$      | $\mathbf{u}$       |
| $\mathbf{f}$ | $\mathbf{t}$ | $\mathbf{f}$ | $\mathbf{t}$ | $\mathbf{t}$      | $\mathbf{t}$       |
| $\mathbf{f}$ | $\mathbf{f}$ | $\mathbf{f}$ | $\mathbf{f}$ | $\mathbf{t}$      | $\mathbf{f}$       |
| $\mathbf{f}$ | $\mathbf{u}$ | $\mathbf{f}$ | $\mathbf{u}$ | $\mathbf{t}$      | $\mathbf{u}$       |
| $\mathbf{u}$ | $\mathbf{t}$ | $\mathbf{u}$ | $\mathbf{t}$ | $\mathbf{t}$      | $\mathbf{u}$       |
| $\mathbf{u}$ | $\mathbf{f}$ | $\mathbf{f}$ | $\mathbf{u}$ | $\mathbf{u}$      | $\mathbf{u}$       |
| $\mathbf{u}$ | $\mathbf{u}$ | $\mathbf{u}$ | $\mathbf{u}$ | $\mathbf{u}$      | $\mathbf{u}$       |

Define  $\text{BoolNot} : D_{\text{Bool}}^\perp \rightarrow D_{\text{Bool}}^\perp$  by  $\text{BoolNot}(\mathbf{t}) = \mathbf{f}$ ,  $\text{BoolNot}(\mathbf{f}) = \mathbf{t}$ , and  $\text{BoolNot}(\mathbf{u}) = \mathbf{u}$ ; write  $\neg a$  for  $\text{BoolNot}(a)$ . Thus false absorbs conjunction, true absorbs disjunction, implication is  $\neg a \vee b$ , and xor is  $(a \vee b) \wedge \neg(a \wedge b)$  on all three values.

**Where.**  $\mathbf{t}$ ,  $\mathbf{f}$ , and  $\mathbf{u}$  abbreviate the two ordinary Boolean values and the typed Boolean bottom.  $\text{BoolOp}_\odot$  is a total typed function for each admitted Boolean operator, and  $\text{BoolNot}$  is its unary counterpart. The table is the complete semantic specification: no short-circuit convention is assumed by the source denotation.

For a resolved role  $r$ , let

$$\tau_r = \begin{cases} \text{Obj}(\text{tgt}(r)), & \text{upper multiplicity one,} \\ \text{Set}(\text{Obj}(\text{tgt}(r))), & \text{otherwise.} \end{cases}$$

Define

$$\begin{aligned} \text{nav}_{\text{obj}} : D_{\text{Obj}(\text{src}(r))}^\perp \times \{r\} \times \text{Qual}_M(r) &\rightarrow D_{\tau_r}^\perp, \\ \text{nav}_{\text{graph}} : \widehat{D}_{\text{Obj}(\text{src}(r))}^\perp \times \{r\} \times \widehat{\text{Qual}}_{\Sigma_G}(R_r) &\rightarrow \widehat{D}_{\tau_r}^\perp. \end{aligned}$$

For a non-bottom source object  $o$ , let  $N_I(o, r, \bar{q}) = \{o' \mid (o, r, o', \bar{q}) \in \text{link}_I\}$ ; for a non-bottom graph object node  $v$ , let  $N_G(v, r, Q) = \{v' \mid \text{Link}_G(v, r, v', Q)\}$ . Each navigation is defined by

$$\text{nav}_{\text{obj}}(o, r, \bar{q}) = \begin{cases} \perp_{\tau_r}, & o = \perp_{\text{Obj}(\text{src}(r))}, \\ o', & u_r = 1 \wedge N_I(o, r, \bar{q}) = \{o'\}, \\ \perp_{\tau_r}, & u_r = 1 \wedge N_I(o, r, \bar{q}) = \emptyset, \\ N_I(o, r, \bar{q}), & u_r \neq 1, \end{cases}$$

and

$$\text{nav}_{\text{graph}}(v, r, Q) = \begin{cases} \hat{\perp}_{\tau_r}, & v = \hat{\perp}_{\text{Obj}(\text{src}(r))}, \\ v', & u_r = 1 \wedge N_G(v, r, Q) = \{v'\}, \\ \hat{\perp}_{\tau_r}, & u_r = 1 \wedge N_G(v, r, Q) = \emptyset, \\ N_G(v, r, Q), & u_r \neq 1. \end{cases}$$

Here  $\text{mult}(r) = [\ell_r, u_r]$ . A reverse surface navigation is first bound to its resolved directed role, so the same equations apply. Instance and graph well-formedness exclude more than one target in the to-one case.

**Where.**  $r$  is a resolved directed role,  $\tau_r$  is its result type determined by the upper multiplicity, and  $N_I$  and  $N_G$  are the source and graph target sets filtered by the same role and qualifier.  $\bar{q}$  and  $Q$  are related by  $\text{encodeQual}$ ;  $u_r = 1$  denotes a to-one role, while any other upper bound uses the finite Set result. The two navigation functions have the same shape, differing only in their source and encoded carriers.

The source clauses are

$$\begin{aligned} \llbracket x \rrbracket_\rho &= \rho(x), \\ \llbracket \ell \rrbracket_\rho &= \ell, \\ \llbracket e.a \rrbracket_\rho &= \text{val}_I(\llbracket e \rrbracket_\rho, a), \\ \llbracket e.r[\bar{q}] \rrbracket_\rho &= \text{nav}_{\text{obj}}(\llbracket e \rrbracket_\rho, r, \llbracket \bar{q} \rrbracket_\rho), \\ \llbracket C.\text{allInstances}() \rrbracket_\rho &= O_C^I, \\ \llbracket \neg e \rrbracket_\rho &= \text{BoolNot}(\llbracket e \rrbracket_\rho), \\ \llbracket \text{Set}\{e_1, \dots, e_k\} \rrbracket_\rho &= \{\text{coerce}_\beta(\llbracket e_i \rrbracket_\rho) \mid 1 \leq i \leq k\}, \\ \llbracket e_1 \odot e_2 \rrbracket_\rho &= \begin{cases} \text{BoolOp}_\odot(\llbracket e_1 \rrbracket_\rho, \llbracket e_2 \rrbracket_\rho), & \odot \in \{\wedge, \vee, \Rightarrow, \text{xor}\}, \\ \text{strict}_{2,\odot}(\llbracket e_1 \rrbracket_\rho, \llbracket e_2 \rrbracket_\rho), & \text{otherwise}, \end{cases} \\ \llbracket \text{let } x = e_1 \text{ in } e_2 \rrbracket_\rho &= \llbracket e_2 \rrbracket_{\rho[x \mapsto \llbracket e_1 \rrbracket_\rho]}, \\ \llbracket \text{if } c \text{ then } t \text{ else } f \rrbracket_\rho &= \begin{cases} \llbracket t \rrbracket_\rho, & \llbracket c \rrbracket_\rho = \text{true}, \\ \llbracket f \rrbracket_\rho, & \llbracket c \rrbracket_\rho = \text{false}, \\ \perp_\tau, & \text{otherwise}. \end{cases} \end{aligned}$$

For a finite Set  $S \in \mathcal{P}_{\text{fin}}(D_\beta^\perp)$  and a typed predicate  $p : D_\beta^\perp \rightarrow D_{\text{Bool}}^\perp$ , define

$$\begin{aligned} \text{Exists}_\beta : \mathcal{P}_{\text{fin}}(D_\beta^\perp) \times (D_\beta^\perp \rightarrow D_{\text{Bool}}^\perp) &\rightarrow D_{\text{Bool}}^\perp, \\ \text{Exists}(S, p) &= \begin{cases} \text{true}, & \exists x \in S. p_x = \text{true}, \\ \text{false}, & \forall x \in S. p_x = \text{false}, \\ \perp_{\text{Bool}}, & \text{otherwise}, \end{cases} \\ \text{ForAll}_\beta : \mathcal{P}_{\text{fin}}(D_\beta^\perp) \times (D_\beta^\perp \rightarrow D_{\text{Bool}}^\perp) &\rightarrow D_{\text{Bool}}^\perp, \\ \text{ForAll}(S, p) &= \begin{cases} \text{false}, & \exists x \in S. p_x = \text{false}, \\ \text{true}, & \forall x \in S. p_x = \text{true}, \\ \perp_{\text{Bool}}, & \text{otherwise}. \end{cases} \end{aligned}$$

where  $p_x$  abbreviates  $p(x)$  for  $x \in S$ ; the displayed cases are quantified over the finite receiver Set and are therefore total on the typed carrier, including the empty Set and the element-bottom token. The iterator syntax is connected to these semantic functions by

$$\begin{aligned} \llbracket S \rightarrow \text{exists}(x \mid p) \rrbracket_\rho &= \text{Exists}(\llbracket S \rrbracket_\rho, v \mapsto \llbracket p \rrbracket_{\rho[x \mapsto v]}), \\ \llbracket S \rightarrow \text{forAll}(x \mid p) \rrbracket_\rho &= \text{ForAll}(\llbracket S \rrbracket_\rho, v \mapsto \llbracket p \rrbracket_{\rho[x \mapsto v]}). \end{aligned}$$

On a collection bottom, all collection operations are typed bottom. On a scalar or to-one arrow receiver,

asSet is applied first. For ordinary finite Sets,

$$\begin{aligned}
\llbracket S \rightarrow \text{select}(x \mid p) \rrbracket_\rho &= \{v \in \llbracket S \rrbracket_\rho \mid \llbracket p \rrbracket_{\rho[x \mapsto v]} = \text{true}\}, \\
\llbracket S \rightarrow \text{reject}(x \mid p) \rrbracket_\rho &= \{v \in \llbracket S \rrbracket_\rho \mid \llbracket p \rrbracket_{\rho[x \mapsto v]} = \text{false}\}, \\
\llbracket S \rightarrow \text{collect}(x \mid b) \rrbracket_\rho &= \{\llbracket b \rrbracket_{\rho[x \mapsto v]} \mid v \in \llbracket S \rrbracket_\rho\}, \\
\llbracket S \rightarrow \text{size}() \rrbracket_\rho &= |\llbracket S \rrbracket_\rho|, \\
\llbracket S \rightarrow \text{isEmpty}() \rrbracket_\rho &= (\llbracket S \rrbracket_\rho = \emptyset), \\
\llbracket S \rightarrow \text{notEmpty}() \rrbracket_\rho &= (\llbracket S \rrbracket_\rho \neq \emptyset), \\
\llbracket v \rightarrow \text{asSet}() \rrbracket_\rho &= \text{asSet}_{D,\beta}(\llbracket v \rrbracket_\rho) \quad \text{for scalar or to-one } v, \\
\llbracket S \rightarrow \text{asSet}() \rrbracket_\rho &= \llbracket S \rrbracket_\rho \quad \text{for a finite-Set receiver.}
\end{aligned}$$

For every admitted conformance witness  $\sigma \leq_M \beta$ , let  $\text{coerce}_{\sigma \Rightarrow \beta} : D_\sigma^\perp \rightarrow D_\beta^\perp$  be the binder-selected coercion, with typed bottom mapped to typed bottom; use  $\text{coerce}_\beta$  when the source type is evident. Let  $\text{eq}_\beta : D_\beta^\perp \times D_\beta^\perp \rightarrow D_{\text{Bool}}^\perp$  be strict equality after both operands are coerced to the common element type  $\beta$ . The remaining Set relations are not left implicit:

$$\begin{aligned}
&\text{Includes}_\beta, \text{Excludes}_\beta : \mathcal{P}_{fin}(D_\beta^\perp) \times D_\beta^\perp \rightarrow D_{\text{Bool}}^\perp, \\
&\text{IncludesAll}_\beta, \text{ExcludesAll}_\beta : \mathcal{P}_{fin}(D_\beta^\perp)^2 \rightarrow D_{\text{Bool}}^\perp, \\
&\text{Union}_\beta, \text{Intersection}_\beta : \mathcal{P}_{fin}(D_\beta^\perp)^2 \rightarrow \mathcal{P}_{fin}(D_\beta^\perp), \\
&\text{IsUnique}_{\beta,\tau} : \mathcal{P}_{fin}(D_\beta^\perp) \times (D_\beta^\perp \rightarrow D_\tau^\perp) \rightarrow D_{\text{Bool}}^\perp, \\
&\text{Includes}(S, a) = \text{Exists}(S, x \mapsto \text{eq}_\beta(x, a)), \\
&\text{Excludes}(S, a) = \neg \text{Includes}(S, a), \\
&\text{IncludesAll}(S, T) = \text{ForAll}(T, y \mapsto \text{Includes}(S, y)), \\
&\text{ExcludesAll}(S, T) = \text{ForAll}(T, y \mapsto \text{Excludes}(S, y)), \\
&\text{Union}(S, T) = \{\text{coerce}_\beta(x) \mid x \in S\} \cup \{\text{coerce}_\beta(y) \mid y \in T\}, \\
&\text{Intersection}(S, T) = \{\text{coerce}_\beta(x) \mid x \in S\} \cap \{\text{coerce}_\beta(y) \mid y \in T\}, \\
&\text{IsUnique}(S, b) = \text{ForAll}(\{(x, y) \in S^2 \mid x \neq y\}, (x, y) \mapsto \neg \text{eq}_\tau(b_x, b_y)).
\end{aligned}$$

The unsubscripted names in the equations abbreviate the corresponding type-indexed functions at the type fixed by the typing derivation;  $b_x$  and  $b_y$  denote the body result at the two bound elements. For  $m \in \{\text{includes}, \text{excludes}, \text{includesAll}, \text{excludesAll}\}$  and  $c \in \{\text{union}, \text{intersection}\}$ , let  $\mathcal{M}_m$  and  $\mathcal{C}_c$  denote the correspondingly named capitalized functions above. The surface-call equations are

$$\begin{aligned}
\llbracket S \rightarrow m(a) \rrbracket_\rho &= \mathcal{M}_m(\llbracket S \rrbracket_\rho, \llbracket a \rrbracket_\rho), \\
\llbracket S \rightarrow c(T) \rrbracket_\rho &= \mathcal{C}_c(\llbracket S \rrbracket_\rho, \llbracket T \rrbracket_\rho), \\
\llbracket S \rightarrow \text{isUnique}(x \mid b) \rrbracket_\rho &= \text{IsUnique}(\llbracket S \rrbracket_\rho, v \mapsto \llbracket b \rrbracket_{\rho[x \mapsto v]}).
\end{aligned}$$

If any Set receiver in these equations is collection bottom, the result is typed bottom. In particular,  $\text{Includes}(\{\perp_\beta\}, a)$  is determined by strict equality and is bottom unless another element supplies a decisive true result. Union and intersection treat  $\perp_\beta$  as an element while keeping  $\perp_{\text{Set}(\beta)}$  distinct from every finite Set.

For a non-bottom object  $o$ ,

$$\begin{aligned}
\text{oclIsKindOf}(o, C) &= (\text{classOf}_I(o) \preceq C), \\
\text{oclAsType}(o, C) &= \begin{cases} o, & \text{classOf}_I(o) \preceq C, \\ \perp_{\text{Obj}(C)}, & \text{otherwise.} \end{cases}
\end{aligned}$$

Both type operations return typed bottom on a bottom receiver. Their surface denotations are

$$\begin{aligned}
\llbracket e. \text{oclIsKindOf}(C) \rrbracket_\rho &= \text{oclIsKindOf}(\llbracket e \rrbracket_\rho, C), \\
\llbracket e. \text{oclAsType}(C) \rrbracket_\rho &= \text{oclAsType}(\llbracket e \rrbracket_\rho, C).
\end{aligned}$$

Appendix A cross-references every surface feature to its typing and semantic schema.

Table 2: Operand, bottom, finite-Set, and coercion policy.

| Family             | Operand/result                                        | Ordinary or empty case                                     | Bottom and $\{\perp\}$ case                                                           | Coercion                                        |
|--------------------|-------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------|
| Scalar             | $D_\sigma \times D_\tau \rightarrow D_\nu$<br>or Bool | Apply the declared<br>primitive operation                  | Any bottom operand<br>yields result bottom                                            | Numeric join or<br>common<br>comparison<br>type |
| Boolean            | $\text{Bool}^2 \rightarrow \text{Bool}$               | Complete table above                                       | Decisive true/false<br>absorbs as displayed                                           | None                                            |
| asSet              | $D_\beta^\perp \rightarrow \text{Set}(\beta)$         | $v \mapsto \{v\}$ ; Set receiver<br>unchanged              | Scalar bottom maps to<br>$\emptyset$ ; collection bottom<br>remains collection bottom | Element type<br>fixed by typing                 |
| Set<br>observation | $\text{Set} \rightarrow \text{Int}/\text{Bool}$       | $ \emptyset  = 0$ ;<br>empty/notEmpty are<br>true/false    | $ \{\perp_\beta\}  = 1$ ; collection<br>bottom yields typed<br>bottom                 | None                                            |
| Set relation       | Set/value or<br>Set/Set to Bool                       | Exists/ForAll<br>equations above                           | Element bottom follows<br>strict equality; collection<br>bottom yields bottom         | Common<br>element type $\beta$                  |
| Set<br>combination | $\text{Set}^2 \rightarrow \text{Set}$                 | Extensional<br>union/intersection;<br>empty identities     | $\{\perp_\beta\}$ is an ordinary finite<br>Set; collection bottom<br>propagates       | Both inputs<br>mapped to<br>common $\beta$      |
| Iterator/type      | Set+body or<br>object+class                           | Empty quantifiers are<br>vacuous; casts use<br>subclassing | Bottom receiver<br>propagates; bottom<br>predicates follow<br>Exists/ForAll tables    | Body join or<br>declared class<br>witness       |

## 4.5 Graph-side denotational clauses

The graph interpretation is not an informal shorthand. It is the following structural interpretation of the normalized carrier. For  $n \in \text{NormExpr}_\tau(M, \Gamma)$  and  $\gamma \in \text{Env}_G(\Gamma)$ , the following clauses apply; in the Set-literal clause each child  $n_i$  has source type  $\sigma_i$  with  $\sigma_i \leq_M \beta$ .

$$\begin{aligned}
 \llbracket x \rrbracket_{\text{graph}}^\gamma &= \gamma(x), \\
 \llbracket \ell \rrbracket_{\text{graph}}^\gamma &= \widehat{\ell}, \\
 \llbracket n.a \rrbracket_{\text{graph}}^\gamma &= \text{read}_G^a(\llbracket n \rrbracket_{\text{graph}}^\gamma), \\
 \llbracket n.r[\bar{Q}] \rrbracket_{\text{graph}}^\gamma &= \text{nav}_{\text{graph}}(\llbracket n \rrbracket_{\text{graph}}^\gamma, r, \llbracket \bar{Q} \rrbracket_{\text{graph}}^\gamma), \\
 \llbracket C.\text{allInstances}() \rrbracket_{\text{graph}}^\gamma &= \text{allInstances}_G(C), \\
 \llbracket \neg n \rrbracket_{\text{graph}}^\gamma &= \widehat{\text{BoolNot}}(\llbracket n \rrbracket_{\text{graph}}^\gamma), \\
 \llbracket \text{Set}\{n_1, \dots, n_k\} \rrbracket_{\text{graph}}^\gamma &= \{\widehat{\text{coerce}}_{\sigma_i \Rightarrow \beta}(\llbracket n_i \rrbracket_{\text{graph}}^\gamma) \mid 1 \leq i \leq k\}, \\
 \llbracket n_1 \odot n_2 \rrbracket_{\text{graph}}^\gamma &= \begin{cases} \widehat{\text{BoolOp}}_\odot(\llbracket n_1 \rrbracket_{\text{graph}}^\gamma, \llbracket n_2 \rrbracket_{\text{graph}}^\gamma), & \odot \in \{\wedge, \vee, \Rightarrow, \text{xor}\}, \\ \widehat{\text{strict}}_{2, \odot}(\llbracket n_1 \rrbracket_{\text{graph}}^\gamma, \llbracket n_2 \rrbracket_{\text{graph}}^\gamma), & \text{otherwise,} \end{cases} \\
 \llbracket \text{let } x = n_1 \text{ in } n_2 \rrbracket_{\text{graph}}^\gamma &= \llbracket n_2 \rrbracket_{\text{graph}}^{\gamma[x \mapsto \llbracket n_1 \rrbracket_{\text{graph}}^\gamma]}, \\
 \llbracket \text{if } c \text{ then } t \text{ else } f \rrbracket_{\text{graph}}^\gamma &= \begin{cases} \llbracket t \rrbracket_{\text{graph}}^\gamma, & \llbracket c \rrbracket_{\text{graph}}^\gamma = \text{true}, \\ \llbracket f \rrbracket_{\text{graph}}^\gamma, & \llbracket c \rrbracket_{\text{graph}}^\gamma = \text{false}, \\ \widehat{\perp}_\tau, & \text{otherwise.} \end{cases}
 \end{aligned}$$

The graph-side operators in this display are typed counterparts of the source operators, not overloaded backend primitives:

$$\begin{aligned}
 \widehat{\text{strict}}_{k, f} : \widehat{D}_{\tau_1}^\perp \times \dots \times \widehat{D}_{\tau_k}^\perp &\rightarrow \widehat{D}_v^\perp, \\
 \widehat{\text{BoolOp}}_\odot : \widehat{D}_{\text{Bool}}^\perp \times \widehat{D}_{\text{Bool}}^\perp &\rightarrow \widehat{D}_{\text{Bool}}^\perp, \\
 \widehat{\text{BoolNot}} : \widehat{D}_{\text{Bool}}^\perp &\rightarrow \widehat{D}_{\text{Bool}}^\perp, \\
 \widehat{\text{eq}}_\beta : \widehat{D}_\beta^\perp \times \widehat{D}_\beta^\perp &\rightarrow \widehat{D}_{\text{Bool}}^\perp, \\
 \widehat{\text{coerce}}_{\sigma \Rightarrow \beta} : \widehat{D}_\sigma^\perp &\rightarrow \widehat{D}_\beta^\perp.
 \end{aligned}$$

Each has the same three-valued table, strict-bottom rule, and declared coercion witness as its source counterpart and must commute with `encodeValue` under A3 and A8.

For graph finite Sets, define the collection-boundary map

$$\text{finiteSet}_{\widehat{D},\beta} : \widehat{D}_{\text{Set}(\beta)}^\perp \rightarrow \mathcal{P}_{fin}(\widehat{D}_\beta^\perp), \quad \text{finiteSet}_{\widehat{D},\beta}(S) = \begin{cases} S, & S \neq \widehat{\perp}_{\text{Set}(\beta)}, \\ \emptyset, & S = \widehat{\perp}_{\text{Set}(\beta)}. \end{cases}$$

The collection operators used below have the following typed signatures:

$$\begin{aligned} \text{Exists}_{G,\beta} &: \mathcal{P}_{fin}(\widehat{D}_\beta^\perp) \times (\widehat{D}_\beta^\perp \rightarrow \widehat{D}_{\text{Bool}}^\perp) \rightarrow \widehat{D}_{\text{Bool}}^\perp, \\ \text{ForAll}_{G,\beta} &: \mathcal{P}_{fin}(\widehat{D}_\beta^\perp) \times (\widehat{D}_\beta^\perp \rightarrow \widehat{D}_{\text{Bool}}^\perp) \rightarrow \widehat{D}_{\text{Bool}}^\perp, \\ \text{Union}_{G,\beta}, \text{Intersection}_{G,\beta} &: \mathcal{P}_{fin}(\widehat{D}_\beta^\perp)^2 \rightarrow \mathcal{P}_{fin}(\widehat{D}_\beta^\perp), \\ \text{IsUnique}_{G,\beta,\tau} &: \mathcal{P}_{fin}(\widehat{D}_\beta^\perp) \times (\widehat{D}_\beta^\perp \rightarrow \widehat{D}_\tau^\perp) \rightarrow \widehat{D}_{\text{Bool}}^\perp. \end{aligned}$$

They are obtained by pointwise encoding of  $\text{Exists}_\beta$ ,  $\text{ForAll}_\beta$ , union/intersection, and  $\text{IsUnique}$ ; their collection-bottom inputs are first completed by  $\text{finiteSet}_{\widehat{D},\beta}$ . For a graph finite Set, let  $\widehat{S} = \text{finiteSet}_{\widehat{D},\beta}(S)$  and let  $\widehat{p}_x$  denote the recursively evaluated body under  $\gamma[x \mapsto v]$ . The collection clauses are

$$\begin{aligned} \llbracket S \rightarrow \text{exists}(x \mid p) \rrbracket_{\text{graph}}^\gamma &= \text{Exists}_{G,\beta}(\widehat{S}, v \mapsto \widehat{p}_x), \\ \llbracket S \rightarrow \text{forAll}(x \mid p) \rrbracket_{\text{graph}}^\gamma &= \text{ForAll}_{G,\beta}(\widehat{S}, v \mapsto \widehat{p}_x), \\ \llbracket S \rightarrow \text{select}(x \mid p) \rrbracket_{\text{graph}}^\gamma &= \{v \in \widehat{S} \mid \widehat{p}_x = \text{true}\}, \\ \llbracket S \rightarrow \text{reject}(x \mid p) \rrbracket_{\text{graph}}^\gamma &= \{v \in \widehat{S} \mid \widehat{p}_x = \text{false}\}, \\ \llbracket S \rightarrow \text{collect}(x \mid b) \rrbracket_{\text{graph}}^\gamma &= \{\llbracket b \rrbracket_{\text{graph}}^{\gamma[x \mapsto v]} \mid v \in \widehat{S}\}, \\ \llbracket S \rightarrow \text{size}() \rrbracket_{\text{graph}}^\gamma &= |\widehat{S}|, \\ \llbracket S \rightarrow \text{isEmpty}() \rrbracket_{\text{graph}}^\gamma &= (\widehat{S} = \emptyset), \\ \llbracket S \rightarrow \text{notEmpty}() \rrbracket_{\text{graph}}^\gamma &= (\widehat{S} \neq \emptyset), \\ \llbracket v \rightarrow \text{asSet}() \rrbracket_{\text{graph}}^\gamma &= \text{asSet}_{\widehat{D},\beta}(\llbracket v \rrbracket_{\text{graph}}^\gamma), \\ \llbracket S \rightarrow \text{union}(T) \rrbracket_{\text{graph}}^\gamma &= \text{Union}_{G,\beta}(\widehat{S}, \widehat{T}), \\ \llbracket S \rightarrow \text{intersection}(T) \rrbracket_{\text{graph}}^\gamma &= \text{Intersection}_{G,\beta}(\widehat{S}, \widehat{T}), \\ \llbracket S \rightarrow \text{isUnique}(x \mid b) \rrbracket_{\text{graph}}^\gamma &= \text{IsUnique}_{G,\beta,\tau}(\widehat{S}, v \mapsto \llbracket b \rrbracket_{\text{graph}}^{\gamma[x \mapsto v]}). \end{aligned}$$

For completeness, graph type tests and casts are typed functions

$$\begin{aligned} \text{kindOf}_G &: \widehat{D}_{\text{Obj}(C')}^\perp \times \mathcal{C} \rightarrow \widehat{D}_{\text{Bool}}^\perp, \\ \text{cast}_G &: \widehat{D}_{\text{Obj}(C')}^\perp \times \mathcal{C} \rightarrow \widehat{D}_{\text{Obj}(C)}^\perp, \end{aligned}$$

with  $\text{kindOf}_G(\widehat{\perp}) = \widehat{\perp}_{\text{Bool}}$ ,  $\text{cast}_G(\widehat{\perp}, C) = \widehat{\perp}_{\text{Obj}(C)}$ ,  $\text{kindOf}_G(v, C) = \text{InstanceOf}_G(v, C)$  for an object node  $v$ , and  $\text{cast}_G(v, C) = v$  exactly when  $\text{InstanceOf}_G(v, C)$  is true (otherwise typed bottom). Hence

$$\begin{aligned} \llbracket n. \text{oclIsKindOf}(C) \rrbracket_{\text{graph}}^\gamma &= \text{kindOf}_G(\llbracket n \rrbracket_{\text{graph}}^\gamma, C), \\ \llbracket n. \text{oclAsType}(C) \rrbracket_{\text{graph}}^\gamma &= \text{cast}_G(\llbracket n \rrbracket_{\text{graph}}^\gamma, C). \end{aligned}$$

Here  $\text{Exists}_{G,\beta}$ ,  $\text{ForAll}_{G,\beta}$ , and the Set operators are the typed graph-carrier functions obtained by the complete three-valued tables and coercion equations above, with  $\widehat{\perp}_{\text{Set}(\beta)}$  distinct from both the empty Set and  $\{\widehat{\perp}_\beta\}$ . Attribute lookup on a bottom receiver, type tests, and casts use the corresponding typed graph bottom equations. Thus every constructor in the normalized grammar has a graph-side clause; no graph evaluation case is left as an untyped backend call.

## 5 Coordinated Transformation and Refinement Contracts

### 5.1 Pipeline

**Context.** Fix an admitted pair  $(C, e) \in \text{Inv}_{\text{val}}(M)$  and its context environment  $\Gamma_C$ . The pipeline below records the successive semantic carriers before any backend execution is discussed.

**Definition 5.1 (Query-interface carrier domains)** *Let*

$$\begin{aligned} \text{COp} = \{ & \text{VariablePlan}, \text{LiteralPlan}, \text{SetPlan}, \\ & \text{AttributeAccessPlan}, \text{NavigationAccessPlan}, \text{CallPlan}, \\ & \text{UnaryPlan}, \text{BinaryPlan}, \text{IfPlan}, \text{LetPlan}, \text{IteratorPlan} \}. \end{aligned}$$

*The typed Cypher query model is the carrier*

$$\text{CQM} \triangleq \left\{ \begin{array}{l} p \mid p \text{ is a finite rooted tree of operators in } \text{COp}, \\ \text{with typed children, lexical scope, aliases, parameters, and a result type} \end{array} \right\}.$$

*Define the typed and row-level interfaces explicitly:*

$$\begin{aligned} \text{Bag}_{\text{fin}}(X) &\triangleq \{ \mu : X \rightarrow \mathbb{N} \mid \text{supp}(\mu) \text{ is finite} \}, \\ \text{Row} &\triangleq \text{Name} \multimap \text{Val}, \quad \text{Rows} \triangleq \text{Bag}_{\text{fin}}(\text{Row}), \\ \text{Rows}_{a:\tau} &\triangleq \{ R \in \text{Rows} \mid \forall r \in \text{supp}(R), \text{dom}(r) = \{a\} \wedge r(a) \in \widehat{D}_\tau^\perp \}. \\ \text{supp} : \text{Rows} &\rightarrow \mathcal{P}_{\text{fin}}(\text{Row}), \quad \text{supp}(R) = \{ r \mid R(r) > 0 \}. \end{aligned}$$

Here  $\text{TypeCQM}$  is the syntax-directed typing predicate for CQM trees; it checks operator signatures, aliases, parameters, scope, and the declared root result type. A row is a finite partial name-to-value map and a row bag retains multiplicity until an explicit Set projection. The operation names in Appendix B are instances of these operator families; they are not additional source-level OCL constructors. Let

$$\text{ParamEnv}(\Delta) \triangleq \{ \pi \mid \pi : \text{Name} \multimap \bigcup_{\tau} \widehat{D}_\tau^\perp, \text{dom}(\pi) = \text{dom}(\Delta) \text{ and } \pi \text{ is type-respecting} \},$$

$$\text{CypherText} \triangleq \text{String},$$

$$\begin{aligned} \text{QueryPlan} &\triangleq \{ q \mid q \text{ is a finite typed target AST with declared parameters, aliases,} \\ &\quad \text{lexical scope, and the root interface required by the wrapper} \}. \end{aligned}$$

$$\text{TypeCQM} : \text{CQM} \rightarrow D_{\text{Bool}}.$$

$\text{TypeCQM}(p)$  holds exactly when the CQM typing, scope, alias, parameter, and root rules hold.

$$\begin{aligned} \text{CYPHER5}_{\text{val}} &\subseteq \text{QueryPlan}, \\ \text{emit}_{\text{Cypher}} &: \text{QueryPlan} \rightarrow \text{CypherText}, \\ \text{parse}_{\text{Cypher}} &: \text{CypherText} \multimap \text{QueryPlan}. \end{aligned}$$

where emission produces concrete text and the partial parser succeeds exactly for text in the selected typed Cypher profile. The round trip  $\text{parse}_{\text{Cypher}}(\text{emit}_{\text{Cypher}}(q)) = q$  is an implementation-refinement obligation, not a second possible type for  $q$ . Let  $\text{DriverRequest} \triangleq \{ (q, \pi) \mid q \in \text{QueryPlan}, \pi \in \text{ParamEnv}(\Delta_q) \}$ . For every declared  $\Gamma$  and  $\Delta$ , the semantic interfaces are

$$\begin{aligned} \text{Eval}_{\text{NVA}} &: \text{AdmittedNVA} \times \text{PG}(\Sigma_G) \times \text{Env}_G(\Gamma) \rightarrow \text{Rows}, \\ \text{Eval}_{\text{CQM}} &: \text{TypedCQM} \times \text{PG}(\Sigma_G) \times \text{Env}_G(\Gamma) \rightarrow \text{Rows}, \\ \text{ExecCypher} &: \text{QueryPlan} \times \text{PG}(\Sigma_G) \times \text{ParamEnv}(\Delta) \rightarrow \text{Rows}, \\ \text{ExecText} &: \text{CypherText} \times \text{PG}(\Sigma_G) \times \text{ParamEnv}(\Delta) \rightarrow \text{Rows}, \end{aligned}$$

If a typed root has result type  $\tau$  and public alias  $a$ , the corresponding evaluation is additionally required to inhabit  $\text{Rows}_{a:\tau}$ ; this root-interface condition is part of  $\text{TypeCQM}$  and of  $\text{SpecPlanAdequacy}$ . Consequently all equalities between  $\text{Eval}_{\text{NVA}}$ ,  $\text{Eval}_{\text{CQM}}$ , and  $\text{ExecCypher}$  compare values in the same row carrier rather than silently coercing scalar, node, or multi-column results. The equations below relate these carriers and do not identify either CQM or  $q$  with raw query text. Equality of  $\text{ExecText}(\text{emit}_{\text{Cypher}}(q), G, \pi)$  and  $\text{ExecCypher}(q, G, \pi)$  is the concrete emission/backend refinement boundary evaluated separately from the theorem.

**Where.** CQM is the finite, rooted, typed operator-tree carrier; it is the intermediate query model rather than raw Cypher text. A row is a finite partial map from names to encoded values, and Rows is a finite bag of such rows, so multiplicity is retained until an explicit support/Set projection. Rows<sub>a:τ</sub> fixes the single public field and its typed value carrier. ParamEnv(Δ) contains exactly the declared, type-respecting parameters; QueryPlan is the single typed target-AST carrier; and DriverRequest pairs a plan with its parameter environment. The four evaluation interfaces therefore share the same row carrier, making equality meaningful rather than a prose-level comparison of unlike results.

**Definition 5.2 (Typed intermediate-tree carriers)** For a finite constructor signature  $K$  with child-type and result-type maps, let

$$\text{Tree}_\tau(K; M, \Gamma) = \left\{ t \mid t \text{ is a finite rooted } K\text{-tree, its root has type } \tau, \right. \\ \left. \text{and every child, binding, and operator application satisfies its declared signature} \right\}.$$

The intermediate carriers are the typed constructor-tree sets

$$\begin{aligned} \text{BoundExpr}_\tau(M, \Gamma) &\triangleq \text{Tree}_\tau(\text{Ctor}_{\text{Bound}}; M, \Gamma), \\ \text{OVA}_\tau(M, \Gamma) &\triangleq \text{Tree}_\tau(\text{Ctor}_{\text{OVA}}; M, \Gamma), \\ \text{NVA}_\tau(M, \Gamma) &\triangleq \text{Tree}_\tau(\text{Ctor}_{\text{NVA}}; M, \Gamma), \\ \text{NVA} &\triangleq \bigcup_{M, \Gamma, \tau} \text{NVA}_\tau(M, \Gamma), \\ \text{AdmittedNVA} &\triangleq \{n \in \text{NVA} \mid \text{ValidNVA}(n)\}, \\ \text{TypedCQM} &\triangleq \{p \in \text{CQM} \mid \text{TypeCQM}(p)\}. \end{aligned}$$

Here  $\text{Ctor}_{\text{Bound}}$ ,  $\text{Ctor}_{\text{OVA}}$ , and  $\text{Ctor}_{\text{NVA}}$  are finite alphabets with explicit signatures for variables, literals, accesses, scalar/Boolean operators, control, Set constructors, iterators, and type operations; the NVA alphabet contains the 26 reachable normalized constructors listed in Appendix B.

$$\begin{aligned} \text{WF}_{\text{NVA}} : \text{NVA} &\rightarrow D_{\text{Bool}}, & \text{NF}_R : \text{NVA} &\rightarrow D_{\text{Bool}}, \\ \text{ValidNVA}(n) &\iff \text{WF}_{\text{NVA}}(n) = \text{true} \wedge \text{NF}_R(n) = \text{true}. \end{aligned}$$

**Where.**  $\text{Tree}_\tau$  is a typed finite syntax-tree carrier rather than a software object. The Bound, OVA, and NVA alphabets are distinct stages; a tree is admitted only when all child types, lexical scopes, and constructor signatures check.  $\text{WF}_{\text{NVA}}$  checks rooted containment, unique identifiers, typing, scope, finite-Set shape, and bottom safety, while  $\text{NF}_R$  excludes every derived redex in the rewrite set  $R$ . Thus  $\text{AdmittedNVA}$  is a predicate-backed set of typed normal forms, and  $\text{TypedCQM}$  is a typed CQM subset rather than a Boolean annotation on an untyped tree.

**Where.**  $\text{SchemaWF}(\Sigma_G)$  constrains declarations before any graph is constructed. The graph predicate constrains a particular finite instance: every label, endpoint, property, key, qualifier, and object identity must be declared and type-compatible. These predicates are the well-formedness obligations used by the two canonical lemmas in Section 8.

The coordinated transformations have the signatures

$$\begin{aligned} T_{\text{SCHEMA}} &: \text{UML}_{WF} \rightarrow \text{GraphSchema}, \\ T_{\text{INSTANCE}}^M &: \text{Inst}_{WF}(M) \rightarrow \text{PG}(T_{\text{SCHEMA}}(M)), \\ T_{\text{CONSTRAINT}}^M &: \text{Inv}_{val}(M) \rightarrow \text{DriverRequest}. \end{aligned}$$

The input to  $T_{\text{CONSTRAINT}}^M$  is the pair  $(C, e)$ ; below we display its two components as separate arguments to keep the context visible. Constraint transformation factors as

$$(C, e) \xrightarrow{T_{\text{BIND}}^M} b \xrightarrow{T_{\text{SEM}}^M} s \xrightarrow{T_{\text{NORM}}^M} n \xrightarrow{T_{\text{CQM}}^M} p \xrightarrow{T_{\text{PLAN}}^M} (q, \pi).$$

For every admitted context  $C$ , environment  $\Gamma_C$ , and result type  $\tau$ , the component signatures are

$$\begin{aligned}
T_{\text{BIND}}^M &: \text{Inv}_{\text{val}}(M, C) \rightarrow \text{BoundExpr}_{\text{Bool}}(M, \Gamma_C), \\
T_{\text{SEM}}^M &: \text{BoundExpr}_{\tau}(M, \Gamma) \rightarrow \text{OVA}_{\tau}(M, \Gamma), \\
T_{\text{NORM}}^M &: \text{OVA}_{\tau}(M, \Gamma) \rightarrow \text{AdmittedNVA} \cap \text{NVA}_{\tau}(M, \Gamma), \\
T_{\text{CQM}}^M &: \text{AdmittedNVA} \rightarrow \text{TypedCQM}, \\
T_{\text{PLAN}}^M &: \text{TypedCQM} \rightarrow \text{DriverRequest}, \\
\text{Realize}^M &\triangleq T_{\text{PLAN}}^M \circ T_{\text{CQM}}^M : \text{AdmittedNVA} \rightarrow \text{DriverRequest}.
\end{aligned}$$

The theorem-bearing transformations are the explicitly named specification maps

$$T_{\text{CQM}}^{M, \text{spec}} \triangleq T_{\text{CQM}}^M, \quad T_{\text{PLAN}}^{M, \text{spec}} \triangleq T_{\text{PLAN}}^M, \quad \text{Realize}^{M, \text{spec}} \triangleq T_{\text{PLAN}}^{M, \text{spec}} \circ T_{\text{CQM}}^{M, \text{spec}}.$$

When the source contract writes  $T_{\text{CQM}}^{\text{spec}}(C, R, n)$ , the context class  $C$  and role registry  $R$  are the metadata carried by  $\text{ctx}_{\text{NVA}}(n)$  and by the typed aliases of  $n$ ; the displayed abbreviation is therefore  $T_{\text{CQM}}^{M, \text{spec}}(C, R, n) \triangleq T_{\text{CQM}}^M(n)$  on that matching context. This preserves the authoritative source signature without introducing a second, semantically different construction. The superscript “spec” marks the normative reference construction; any concrete realization is related to it only by a separate refinement argument. The factorization is semantically significant. Binding resolves model symbols and types; semantic abstraction removes surface-specific detail; normalization reduces derived operations to a closed validation algebra; and realization builds a typed target plan plus parameters; concrete Cypher text is produced only by the subsequent emitter.

## 5.2 Concrete schema and instance mappings

The canonical schema transformation is defined extensionally:

$$\begin{aligned}
\mathcal{N}_T &= \{N_C \mid C \in \mathcal{C}\}, & \mathcal{R}_T &= \{R_r \mid r \in \mathcal{R}\}, \\
\mathcal{P}_D &= \{p_{id}^M\} \cup \{p_a \mid a \in \mathcal{A}\} \cup \{p_{r,i}^q \mid q_i \in \text{qual}(r)\}, \\
\text{src}_T(R_r) &= N_{\text{src}(r)}, & \text{tgt}_T(R_r) &= N_{\text{tgt}(r)}, \\
\text{owner}_P(p_{id}^M) &= \{N_C \mid C \in \mathcal{C}\}, & \text{type}_P(p_{id}^M) &= \text{ObjectId}, & (\text{SMap}) \\
\text{owner}_P(p_a) &= \{N_C \mid C \preceq \text{owner}(a)\}, & \text{type}_P(p_a) &= \text{type}_A(a), \\
\text{owner}_P(p_{r,i}^q) &= \{R_r\}, & \text{type}_P(p_{r,i}^q) &= \text{type}(q_{r,i}), \\
\mathcal{K}_D &= \{(N_C, p_{id}^M) \mid C \in \mathcal{C}\}.
\end{aligned}$$

$\text{meta}_T$  records the injective maps  $C \mapsto N_C$ ,  $a \mapsto p_a$ ,  $r \mapsto R_r$ , inheritance closure, opposite-role identity, model scope, multiplicity, ordered qualifier keys, and the object-label set  $\text{ObjLabels}(\Sigma_G) = \{N_C \mid C \in \mathcal{C}\}$ . Thus

$$\text{ObjLabels} : \text{GraphSchema} \rightarrow \mathcal{P}_{\text{fin}}(\mathcal{N}_T)$$

is the schema-indexed set of labels denoting model objects. No two model elements in one scope map to the same validation-visible symbol.

**Where.**  $N_C$ ,  $R_r$ ,  $p_a$ , and  $p_{r,i}^q$  are fresh target symbols for a class, role, attribute, and qualifier declaration.  $\text{owner}_P$  is set-valued so an inherited attribute property is readable on every subtype node; a qualifier property is owned only by its relationship type. The key set  $\mathcal{K}_D$  makes  $p_{id}^M$  unique per object label, while  $\text{meta}_T$  records the inverse correspondence needed by the abstract graph observers.

For  $I \in \text{Inst}_{WF}(M)$ , the canonical instance mapping is

$$\begin{aligned}
V_{\text{obj}} &= \{n_o \mid o \in O\}, & \text{label}_V(n_o) &= N_{\text{classOf}_I(o)}, \\
\text{prop}(n_o, p_{id}^M) &= \text{id}_I(o), & \text{prop}(n_o, p_a) &= \text{encodeValue}(\text{val}_I(o, a)), \\
E &= \{e_{o,r,o',\bar{q}} \mid (o, r, o', \bar{q}) \in \text{link}_I\}, \\
\text{src}_G(e_{o,r,o',\bar{q}}) &= n_o, & \text{tgt}_G(e_{o,r,o',\bar{q}}) &= n_{o'}, \\
\text{label}_E(e_{o,r,o',\bar{q}}) &= R_r, & \text{prop}(e_{o,r,o',\bar{q}}, p_{r,i}^q) &= \text{encodeValue}_{\text{type}(q_{r,i})}(q_i), \\
\text{qualifier}(e_{o,r,o',\bar{q}}) &= Q_{r,\bar{q}}^M. & & (\text{IMap})
\end{aligned}$$

Only applicable attributes are materialized; typed bottom uses the declared separated representation. SMap and IMap create no additional validation-visible object, type, property value, semantic link, or class membership.

**Where.**  $n_o$  is the graph node corresponding to source object  $o$  and  $e_{o,r,o',\bar{q}}$  is the edge corresponding to the source link tuple.  $\text{encodeValue}$  preserves declared primitive types and separated bottoms;  $Q_{r,\bar{q}}^M$  preserves qualifier order and role identity. IMap materializes only observations in the formal source state, so persistence metadata cannot create a semantic witness.

### 5.3 Bound-to-semantic constructor map

$T_{\text{SEM}}$  erases surface spelling while retaining resolved symbols, types, multiplicities, collection shapes, and lexical identifiers. It is homomorphic on children. Its exhaustive families are variables/literals, attribute/navigation/extent, scalar and Boolean operations, if/let, Set construction and operations, iterators/collect, cardinality/emptiness, and type operations. Admission excludes any unrepresented constructor.

### 5.4 Relational refinement interface

Let  $e, b, s, n, p, q, \pi$  denote a surface body, bound tree, OVA tree, NVA tree, CQM plan, query, and parameter environment. Write  $\gamma \bowtie_q \pi$  when live aliases, parameter values, lexical scopes, and declared names correspond. Its carrier and the adjacent relations are

$$\begin{aligned} \bowtie_q &\subseteq \text{Env}_G(\Gamma) \times \text{ParamEnv}(\Delta_q), \\ \text{Ref}_{\text{ParseAdmit}} &\subseteq \text{OCLExpr}_\tau(M, \Gamma) \times \text{BoundExpr}_\tau(M, \Gamma), \\ \text{Ref}_{\text{OVA}, \text{NVA}} &\subseteq \text{OVA}_\tau(M, \Gamma) \times \text{NVA}_\tau(M, \Gamma), \\ \text{Ref}_{\text{NVA}, \text{CQM}} &\subseteq \text{NVA}_\tau(M, \Gamma) \times \text{TypedCQM}, \\ \text{Ref}_{\text{CQM}, \text{Cypher}} &\subseteq \text{TypedCQM} \times \text{QueryPlan} \times \text{ParamEnv}(\Delta_q). \end{aligned}$$

For every  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $\Gamma \in \text{TypeEnv}$ , type  $\tau$ , well-typed terms in these carriers, and corresponding  $\rho \in \text{Env}_I(\Gamma)$ , the source relations obey

$$\begin{aligned} \text{Ref}_{\text{ParseAdmit}}(e, b) &\Rightarrow \llbracket e \rrbracket_{\text{OCL}}^{M, I, \rho} = \llbracket b \rrbracket_{\text{src}}^{M, I, \rho}, \\ \text{Ref}_{\text{OVA}, \text{NVA}}(s, n) &\Rightarrow \llbracket s \rrbracket_{\text{src}}^{M, I, \rho} = \llbracket n \rrbracket_{\text{src}}^{M, I, \rho}. \end{aligned}$$

For every schema  $\Sigma_G$ ,  $G \in \text{PG}(\Sigma_G)$ , and  $\gamma \in \text{Env}_G(\Gamma)$  satisfying **SchemaWF** and **GraphWF**,

$$\begin{aligned} \text{Ref}_{\text{NVA}, \text{CQM}}(n, p) &\Rightarrow \text{Eval}_{\text{NVA}}(n, G, \gamma) = \text{Eval}_{\text{CQM}}(p, G, \gamma), \\ \text{Ref}_{\text{CQM}, \text{Cypher}}(p, q, \pi) \wedge \gamma \bowtie_q \pi &\Rightarrow \text{Eval}_{\text{CQM}}(p, G, \gamma) = \text{ExecCypher}(q, G, \pi). \end{aligned}$$

Thus every equality includes its model, environment, and quantification; no Boolean flag is accepted as a refinement witness.

### 5.5 Normalization rules

Every rewrite preserves type, lexical scope, finite-Set shape, bottom policy, and source denotation. Representative derived-operation rewrites are

$$\begin{aligned} \text{ForAll}(S, x, p) &\rightsquigarrow \neg \text{Exists}(S, x, \neg p), \\ p &\Rightarrow q \rightsquigarrow \neg p \vee q, \\ p \text{ xor } q &\rightsquigarrow (p \vee q) \wedge \neg(p \wedge q), \\ \text{IsEmpty}(S) &\rightsquigarrow \neg \text{Exists}(S, x, \text{true}), \\ \text{NotEmpty}(S) &\rightsquigarrow \text{Exists}(S, x, \text{true}), \\ \text{Size}(S) &\rightsquigarrow \text{Count}(S), \\ \text{Reject}(S, x, p) &\rightsquigarrow \text{Select}(S, x, \neg p), \\ \text{Includes}(S, a) &\rightsquigarrow \text{Exists}(S, x, x = a), \\ \text{Excludes}(S, a) &\rightsquigarrow \neg \text{Exists}(S, x, x = a). \end{aligned}$$

Here `Count` is a primitive NVA/CQM cardinality constructor introduced by normalization. It is not an additional surface feature of  $\text{OCL}_{\text{val}}$ ; its denotation is the finite cardinality of the normalized Set. Its stage-specific signatures are

$$\text{rootType} : \text{CQM} \rightarrow \text{Type}, \quad \text{CQM}_\tau \triangleq \{p \in \text{TypedCQM} \mid \text{rootType}(p) = \tau\},$$

and

$$\text{Count}_{\text{NVA},\beta} : \text{NVA}_{\text{Set}(\beta)}(M, \Gamma) \rightarrow \text{NVA}_{\text{Int}}(M, \Gamma), \quad \text{CountExpr}_\beta : \text{CQM}_{\text{Set}(\beta)} \rightarrow \text{CQM}_{\text{Int}}.$$

The notation  $\text{Count}(S)$  abbreviates the first constructor and  $\text{CountExpr}(S)$  its typed CQM expansion; neither introduces a new surface-level OCL operation. In a local realization equation,  $\text{CountExpr}(S)$  means  $\text{CountExpr}_\beta(T_{\text{CQM}}(S))$  after the receiver plan has been constructed.

**Where.** `Count` belongs to the normalized algebra and the typed query model, not to the surface constructor alphabet  $\text{Ctor}_{\text{OCL}_{\text{val}}}$ .  $S$  is a finite-Set view and its cardinality is an integer. The rewrite is admissible only when the receiver's type and coercion witness establish that view. The remaining derived Set relations are

$$\begin{aligned} \text{IncludesAll}(S, T) &\rightsquigarrow \neg \text{Exists}(T, y, \neg \text{Exists}(S, x, x = y)), \\ \text{ExcludesAll}(S, T) &\rightsquigarrow \neg \text{Exists}(T, y, \text{Exists}(S, x, x = y)). \end{aligned}$$

Freshness of  $x$  and  $y$  and existence of the element-type coercion join are side conditions. Every constructor not on the derived-operation list is lifted structurally after its children are normalized:

$$\begin{aligned} \text{Attr}(e, a) &\rightsquigarrow \text{Attr}(T_{\text{NORM}}(e), a), \\ \text{Compare}(k, l, r) &\rightsquigarrow \text{Compare}(k, T_{\text{NORM}}(l), T_{\text{NORM}}(r)), \\ \text{Select}(S, x, p) &\rightsquigarrow \text{Select}(T_{\text{NORM}}(S), x, T_{\text{NORM}}(p)). \end{aligned}$$

Semantic equivalence is

$$u \simeq_{M, \Gamma, \tau} u' \iff \forall I \in \text{Inst}_{WF}(M), \forall \rho \in \text{Env}_I(\Gamma) : \llbracket u \rrbracket_{\text{src}}^{M, I, \rho} = \llbracket u' \rrbracket_{\text{src}}^{M, I, \rho}.$$

Every normalization rule must establish  $u \rightsquigarrow u' \Rightarrow u \simeq_{M, \Gamma, \tau} u'$ . Let  $\text{MSet}_{\text{fin}}(\mathbb{N})$  denote the carrier of finite multisets over natural-number constructor ranks, and let

$$\begin{aligned} \text{derivedRankMultiset} &: \text{OVA} \rightarrow \text{MSet}_{\text{fin}}(\mathbb{N}), \\ \text{unprocessedChildren} &: \text{OVA} \rightarrow \mathbb{N}, \\ \text{treeSize} &: \text{OVA} \rightarrow \mathbb{N} \end{aligned}$$

return, respectively, the multiset of ranks of remaining derived constructors, the number of children not yet normalized, and the number of tree nodes. Define

$$\mu(u) = (\text{derivedRankMultiset}(u), \text{unprocessedChildren}(u), \text{treeSize}(u))$$

with the multiset extension of the strict constructor-rank order followed by lexicographic order. A derived rewrite removes its root rank and introduces only lower-rank constructors; a lifting step decreases the number of unprocessed children. Hence every step decreases  $\mu$ .

Local proofs use De Morgan duality for quantifiers, exhaustive three-valued Boolean cases, finite-Set cardinality/membership identities, fresh variables, the declared common element coercion, and congruence for structural lifting.

## 5.6 NVA-to-CQM constructor coverage

The NVA-to-CQM transformation is partial outside the admitted NVA grammar and total inside it:

$$T_{\text{CQM}}^M : \text{AdmittedNVA} \rightarrow \text{TypedCQM}.$$

Every reachable concrete NVA constructor has exactly one rule in Appendix B. Each local rule carries denotation, parameter, alias, lexical-scope, multiplicity, collection-shape, and bottom obligations. The composed theorem has the shape

$$\text{Eval}_{\text{NVA}}(n, G, \gamma) = \text{Eval}_{\text{CQM}}(T_{\text{CQM}}^M(n), G, \gamma),$$

provided the displayed constructor side conditions hold. The equations are proved below for the normative rules rather than assumed as a hidden field of the transformed object.

## 5.7 CQM-to-query-plan realization

The typed-plan realization preserves aliases, lexical scope, parameters, freshness, role direction and qualifiers, finite-Set distinctness, separated bottoms, branch/aggregation/filter behavior, and the root predicate *not validation true* followed by Set-distinct *useId* projection. The later emitter produces Cypher text; relating execution of that text to the plan denotation is an implementation refinement outside the theorem.

For every row  $r$  in Appendix B, the local NVA-to-CQM obligation is a quantified relation, not merely its LR name. Its interface is

$$\text{LR}_r \subseteq \text{AdmittedNVA} \times \text{TypedCQM}, \quad \text{Pre}_r \subseteq \text{AdmittedNVA} \times \text{Env}_G(\Gamma).$$

Equivalently,  $\text{Pre}_r$  is a typed predicate  $\text{AdmittedNVA} \times \text{Env}_G(\Gamma) \rightarrow D_{\text{Bool}}$ . The relation is defined by

$$\begin{aligned} \text{LR}_r(n, p) \iff \forall \Sigma_G \in \text{GraphSchema}, \forall G \in \text{PG}(\Sigma_G), \\ \forall \gamma \in \text{Env}_G(\Gamma). \\ \text{SchemaWF}(\Sigma_G) \wedge \text{GraphWF}(\Sigma_G, G) \wedge \text{Pre}_r(n, \gamma) \\ \Rightarrow \text{Eval}_{\text{NVA}}(n, G, \gamma) = \text{Eval}_{\text{CQM}}(p, G, \gamma), \end{aligned}$$

where  $p = T_{\text{CQM}}^M(n)$  and  $\text{Pre}_r$  contains the row's type, scope, multiplicity, parameter, collection-shape, and bottom premises. The CQM-to-plan obligation is typed by

$$\text{CY} \subseteq \left\{ (p, q, \pi) \mid p \in \text{TypedCQM}, q \in \text{QueryPlan}, \pi \in \text{ParamEnv}(\Delta_q) \right\}$$

and defined by

$$\begin{aligned} \text{CY}(p, q, \pi) \iff (q, \pi) = T_{\text{PLAN}}^{M, \text{spec}}(p) \wedge \\ \forall \Sigma_G, \forall G \in \text{PG}(\Sigma_G), \forall \Gamma \in \text{TypeEnv}, \forall \gamma \in \text{Env}_G(\Gamma). \\ (\text{SchemaWF}(\Sigma_G) \wedge \text{GraphWF}(\Sigma_G, G) \wedge \text{RowCorr}_\Gamma(\gamma, \pi)) \Rightarrow, \\ \text{ExecCypher}(q, G, \pi) = \text{Eval}_{\text{CQM}}(p, G, \gamma) \end{aligned}$$

at the plan's declared result interface.

## 5.8 Constructor-local realization obligations

The relation names  $\text{LR}_r$  and  $\text{CY}$  are not proof results by themselves. For a row projection  $a$  define

$$\text{SemSet}_a : \text{Rows}_{a:\tau} \rightarrow \mathcal{P}_{fin}(\widehat{D}_\tau^\perp), \quad \text{SemSet}_a(R) = \{r(a) \mid r \in \text{supp}(R)\},$$

and let  $\text{RowCorr}_\Gamma(\gamma, \pi)$  mean that the live aliases and declared parameters have equal typed values, scopes, and fresh names.  $\text{Rows}_{a:\tau}$  is the subcarrier whose rows expose exactly one field  $a$  of type  $\tau$ . Formally,

$$\text{RowCorr}_\Gamma : \text{Env}_G(\Gamma) \times \text{ParamEnv}(\Delta_q) \rightarrow D_{\text{Bool}}.$$

The following equations are the local proof obligations consumed by L5; each is quantified over every  $\Sigma_G$ ,  $G \in \text{PG}(\Sigma_G)$ , and well-typed environment satisfying  $\text{SchemaWF}$ ,  $\text{GraphWF}$ , and  $\text{RowCorr}_\Gamma$ .

Table 3: Constructor-local realization equations aligned with the normative proof source.

| Obligation          | Required equality                                                                                                                                                           | Status boundary                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| LR-Variable/Literal | For $p_x = T_{\text{CQM}}^{M, \text{spec}}(x)$ , $\text{rootVal}(\text{Eval}_{\text{CQM}}(p_x, G, \gamma)) = \gamma(x)$ ; a literal plan returns its typed encoded literal. | typing, parameter coverage, C1  |
| LR-Attribute        | Attribute access returns $\text{read}_G^a(v)$ for the recursively realized receiver $v$ ; non-applicable receivers do not enter a property lookup.                          | R4, CY1, CY3, bottom separation |

| Obligation        | Required equality                                                                                                                                                                                                                                                                                        | Status boundary                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| LR-Navigation     | For ordered tuple $\bar{q}$ , the plan denotes exactly $\text{nav}_{\text{graph}}(v, r, Q_{r,\bar{q}}^M)$ ; direction, role identity, and qualifier order are retained.                                                                                                                                  | R5–R6, CY1–CY2                   |
| LR-Boolean/Scalar | Each typed unary or binary plan applies the declared three-valued Boolean or scalar table to the recursively realized operands, including short-circuit and typed-bottom cases.                                                                                                                          | A3, A8, CY3–CY4                  |
| LR-Quantifier     | $\text{boolG}(\text{rootVal}_{\text{value},\text{Bool}}(\text{Eval}_{\text{CQM}}(p_{\exists}, G, \gamma))) = \text{true}$ iff some $v \in \text{finiteSet}_{\widehat{D}}(\llbracket S \rrbracket_{\text{graph}}^{\gamma})$ makes the graph body true; the universal case is absence of a counterexample. | C1, C4, CY4                      |
| LR-Select         | $\text{SemSet}_a(\text{Eval}_{\text{CQM}}(p_{\text{select}}, G, \gamma)) = \{v \in \widehat{S} \mid \llbracket p \rrbracket_{\text{graph}}^{\gamma[x \mapsto v]} = \text{true}\}$ .                                                                                                                      | C1, CY3, CY6                     |
| LR-Collect        | $\text{SemSet}_a(\text{Eval}_{\text{CQM}}(p_{\text{collect}}, G, \gamma)) = \{\llbracket b \rrbracket_{\text{graph}}^{\gamma[x \mapsto v]} \mid v \in \widehat{S}\}$ .                                                                                                                                   | Set-image, bottom token, CY5–CY6 |
| LR-Membership     | The generated membership predicate is true iff the element, after the declared common-type coercion, belongs to $\text{finiteSet}_{\widehat{D}}(\llbracket S \rrbracket_{\text{graph}}^{\gamma})$ .                                                                                                      | typed equality and CY4           |
| LR-SetRelation    | Subset and disjointness plans respectively denote $T \subseteq S$ and $T \cap S = \emptyset$ after both Sets are lifted to their declared common type.                                                                                                                                                   | C5, coercion commutation         |
| LR-Cardinality    | $\text{rootVal}_{\text{value},\text{Int}}(\text{Eval}_{\text{CQM}}(\text{CountExpr}(S), G, \gamma)) =  \text{finiteSet}_{\widehat{D}}(\llbracket S \rrbracket_{\text{graph}}^{\gamma}) $ ; emptiness is the corresponding zero/non-zero equation.                                                        | CY4–CY6                          |
| LR-TypeOps        | Kind-of and cast plans agree with $\text{InstanceOf}_G$ and the typed cast; a bottom receiver never enters a node pattern.                                                                                                                                                                               | R3, CY1, CY3, CY9                |
| LR-Conditional    | Exactly one guarded branch contributes the graph value selected by $\text{vtrue}_G$ of the condition; Set branches are projected extensionally.                                                                                                                                                          | C1, CY3, CY9                     |
| LR-Let            | The initializer is evaluated once, embedded at the declared type, and the body is evaluated under $\gamma[x \mapsto v]$ with outer aliases restored afterwards.                                                                                                                                          | scope, freshness, C1             |

**Lemma 5.3 (Normative constructor realization)** *Let  $n \in \text{AdmittedNVA}$  and  $p = T_{\text{CQM}}^{M,\text{spec}}(n)$ . If every reachable constructor instance satisfies its corresponding syntactic side condition  $\text{Pre}_r$ , then its  $\text{LR}_r(n, p)$  equation in Table 3 holds.*

*Proof.* Proceed by structural induction on the admitted NVA derivation. Variables and literals follow from typed environment lookup and literal embedding. Attributes and qualified navigation unfold to  $\text{read}_G$  and  $\text{nav}_{\text{graph}}$  with the same role and ordered tuple. Boolean/scalar nodes apply the declared typed tables to the child induction hypotheses. Select uses finite-set restriction; exists and forAll use witness and counterexample semantics; cardinality uses support after distinct projection. Membership, Set relations, and collect use extensional finite-set semantics. Type tests/casts unfold R3. Finally, let extends the corresponding environment with a fresh binding, while if evaluates exactly the branch selected by the condition. These cases exhaust the normalized grammar, so no semantic-preservation premise or default constructor case remains.  $\square$

**Lemma 5.4 (SpecPlanSim sound)** *Let  $n \in \text{AdmittedNVA}$ ,  $p = T_{\text{CQM}}^{M,\text{spec}}(n)$ , and  $(q, \pi) = T_{\text{PLAN}}^{M,\text{spec}}(p)$ . If every reachable constructor instance satisfies A6-LR’s syntactic side conditions and  $\text{CY}(p, q, \pi)$  holds, then for every well-formed schema/graph and corresponding environment,*

$$\text{Eval}_{\text{NVA}}(n, G, \gamma) = \text{Eval}_{\text{CQM}}(p, G, \gamma),$$

and, whenever  $\text{RowCorr}_{\Gamma}(\gamma, \pi)$  holds,

$$\text{Eval}_{\text{CQM}}(p, G, \gamma) = \text{ExecCypher}(q, G, \pi).$$

*Proof.* The first equality follows from Lemma 5.3; its induction consumes only the stated side conditions and child hypotheses. The second equality is the displayed CY equation at the typed root; its environment and parameter premise is exactly RowCorr<sub>Γ</sub>. Thus the simulation derives constructor-local preservation rather than assuming the composed result equality.  $\square$

The equations above are theorem conclusions for the normative rules, not an assumption that a transformed object contains a field called `correct`. Any concrete realization is related to the normative rules only through an explicit structural refinement certificate.

The CQM and typed query plan are normative for this interface. An emitted Cypher string or optimized backend request is realization-specific until related to the plan by an explicit refinement argument. Accordingly define

$$\begin{aligned} \text{ctx}_{\text{NVA}} : \text{AdmittedNVA} &\rightarrow \mathcal{C}, \\ \text{SpecPlanAdequacy}(M, C, n, p, q, \pi, G) &\iff \text{ValidNVA}(n) \wedge \text{ctx}_{\text{NVA}}(n) = C \\ &\wedge p \in \text{TypedCQM} \wedge (q, \pi) = T_{\text{PLAN}}^{M, \text{spec}}(p) \\ &\wedge q \in \text{CYPHER5}_{\text{val}} \wedge \text{parse}_{\text{Cypher}}(\text{emit}_{\text{Cypher}}(q)) = q \\ &\wedge \text{typing, closure, alias, parameter, result-kind, and} \\ &\text{root-interface obligations hold.} \end{aligned}$$

The returned-ID equality is the conclusion of the constructor induction (L5), not a field of SpecPlanAdequacy and not a premise smuggled into CY.

## 6 Graph Adequacy and the Canonical Encoder

**Notation.** Let  $\eta_{M,I}$  map source objects to object nodes in a graph over  $T_{\text{SCHEMA}}(M)$ . The adequacy clauses below are observations of this map, not properties of a particular storage API.

**Definition 6.1 (Graph adequacy)** Let  $\eta_{M,I} : O \rightarrow V_{\text{obj}}(G)$  be a candidate object–node correspondence. Then

$$\text{GraphAdequate}(M, I, G, \eta_{M,I}) \iff \bigwedge_{i=1}^7 R_i,$$

where every clause ranges over applicable well-typed objects, classes, attributes, roles, and qualifier tuples:

**R1 — stable identity.**  $\text{prop}(\eta(o), p_{\text{id}}^M) = \text{id}_I(o)$  and  $\eta(o_1) = \eta(o_2) \iff o_1 = o_2$ .

**R2 — object coverage.**  $\eta : O \rightarrow V_{\text{obj}}(G)$  is a bijection and every validation-visible object node has a unique source representative.

**R3 — typing.**  $\text{classOf}_I(o) \preceq C$  iff  $\text{InstanceOf}_G(\eta(o), C)$ .

**R4 — attributes.**  $\text{read}_G^a(\eta(o)) = \text{encodeValue}_{\text{type}_A(a)}(\text{val}_I(o, a))$ .

**R5 — links.**  $(o, r, o', \bar{q}) \in \text{link}_I$  iff  $\text{Link}_G(\eta(o), r, \eta(o'), Q_{r, \bar{q}}^M)$ .

**R6 — navigation.**  $\text{encodeValue}_{\tau_r}(\text{nav}_{\text{obj}}(o, r, \bar{q})) = \text{nav}_{\text{graph}}(\eta(o), r, Q_{r, \bar{q}}^M)$ .

**R7 — class extent.**  $\eta[O_C^I] = \text{allInstances}_G(C)$ .

**Where.**  $\eta_{M,I}$  is the candidate object-to-node map,  $R_1$ – $R_7$  are the stable-identity, coverage, typing, attribute, link, navigation, and extent clauses, respectively, and  $Q_{r, \bar{q}}^M$  is the encoded qualifier tuple. The quantification is over all applicable source declarations and values, so adequacy compares observations rather than merely the number of graph nodes or rows. The relation is a predicate on  $(M, I, G, \eta)$  and is independent of the concrete Cypher text realization.

R1 and R2 prevent identity loss, duplication, and unobservable source objects. R3 and R7 align context selection. R4–R6 align the atomic observations used by structural induction. Row-count agreement cannot replace these clauses: two unequal identifier Sets may have the same size.

**Definition 6.2 (Canonical instance realization)** For  $M \in \text{UML}_{WF}$  and  $I \in \text{Inst}_{WF}(M)$ , create one distinct object node  $n_o$  for each  $o \in O$ , let  $\eta(o) = n_o$ , and impose

$$\begin{aligned} V_{\text{obj}}(G) &= \{n_o \mid o \in O\}, & \text{prop}(n_o, p_{id}^M) &= \text{id}_I(o), \\ \text{InstanceOf}_G(n_o, C) &\iff \text{classOf}_I(o) \preceq C, \\ \text{read}_G^a(n_o) &= \text{encodeValue}_{\text{type}_A(a)}(\text{val}_I(o, a)), \\ \text{Link}_G(n_o, r, n_{o'}, Q_{r,\bar{q}}^M) &\iff (o, r, o', \bar{q}) \in \text{link}_I. \end{aligned} \tag{E1}$$

No additional validation-visible object or semantic-link fact is created.

**Where.** The canonical realization is the particular encoder used in the theorem:  $n_o$  is one fresh node for each source object,  $e_{o,r,o',\bar{q}}$  is one edge for each source link, and  $\eta(o) = n_o$ . Equations (E1) fix the identity, dynamic type, attribute, role, direction, and qualifier observations; they do not describe a concrete serialization format or transaction behavior.

**Lemma 6.3 (Canonical schema well-formedness)** For every  $M \in \text{UML}_{WF}$ ,

$$\text{SchemaWF}(T_{\text{SCHEMA}}(M)).$$

*Proof.* Finiteness transfers from  $\mathcal{C}, \mathcal{A}, \mathcal{R}$  to their injective SMap images and keys. SMap assigns every relationship its declared endpoints and every identity, attribute, and qualifier property its owner and type;  $\text{meta}_T$  preserves scope, inheritance, and qualifier order. These clauses exhaust **SchemaWF**.  $\square$

**Lemma 6.4 (Canonical instance well-formedness)** For every canonical source configuration, let

$$M \in \text{UML}_{WF}, \quad I \in \text{Inst}_{WF}(M), \quad G = T_{\text{INSTANCE}}^M(I).$$

Then

$$\text{GraphWF}(T_{\text{SCHEMA}}(M), G).$$

*Proof.* The sets  $V = \{n_o \mid o \in O\}$  and  $E = \{e_{o,r,o',\bar{q}} \mid (o, r, o', \bar{q}) \in \text{link}_I\}$  are finite. IMap gives each edge declared endpoints, label, and typed qualifiers; SMap declares all used labels and properties. Source endpoint, multiplicity, opposite, uniqueness, and non-dangling clauses transfer to the edges, while injective  $\text{id}_I$  gives unique  $p_{id}^M$  values. No extra validation-visible element is created; hence **GraphWF**.  $\square$

**Theorem 6.5 (Canonical encoder adequacy)** If  $\text{UMLWF}(M)$  and  $\text{InstWF}(M, I)$  hold, realization (E1) satisfies  $\text{GraphAdequate}(M, I, T_{\text{INSTANCE}}^M(I), \eta_{M,I})$ .

*Proof.* Lemmas 6.3 and 6.4 make all graph observers in R1–R7 well defined. IMap gives R1; its one-node-per-object construction and no-extra-node clause give the R2 bijection. Inheritance metadata gives R3, and attribute encoding gives R4. The one-edge-per-link construction, with direction and injective qualifier encoding, gives both directions of R5. R6 then follows by the to-one/to-many navigation cases, and R7 follows from R2 and R3 by the two extent inclusions.  $\square$

An alternative encoding need not use the same labels or physical relationships, but it must prove all seven observational clauses.

## 7 Preservation Criterion and Assumptions

**Context.** Fix an admitted pair  $(C, e)$ , a conforming state  $I$ , and the graph/query representations produced by the transformations. The criterion compares only stable model-scoped identifiers.

For a context object  $o$ , let  $\rho_o = [\text{self} \mapsto o]$ .

**Definition 7.1 (Source violation IDs)** For admitted invariant  $e$  in context  $C$ ,

$$\begin{aligned} \text{ViolIds}_{\text{OCL}}(e, C, M, I) &= \{\text{id}_I(o) \mid o \in O_C^I, \\ &\quad \text{vtrue}_{\text{OCL}}(\llbracket e \rrbracket_{\text{OCL}}^{M,I,\rho_o}) = \text{false}\}. \end{aligned}$$

**Where.**  $O_C^I$  is the context extent,  $\rho_o$  binds the distinguished variable **self** to  $o$ , and  $\text{vtrue}_{\text{OCL}}$  maps exactly the ordinary value **true** to validation success. Thus the set contains one stable identifier for each context object whose admitted invariant is not true; a typed bottom is included because it is validation-false under the declared policy.

**Definition 7.2 (Returned stable violation IDs)** For target query  $q$ , graph  $G$ , and parameters  $\pi$ ,

$$\text{ReturnedIds}(q, G, \pi) = \text{useIdSet}(\text{ExecCypher}(q, G, \pi)),$$

with the well-typed result requirement

$$\text{ExecCypher}(q, G, \pi) \in \text{Rows}_{\text{useId:ObjectId}};$$

if this requirement fails,  $\text{ReturnedIds}$  is undefined rather than being obtained by coercing an arbitrary row shape. where the root-interface projection has signature

$$\text{useIdSet} : \text{Rows}_{\text{useId:ObjectId}} \rightarrow \mathcal{P}_{\text{fin}}(\text{ObjectId}), \quad \text{useIdSet}(R) = \{r(\text{useId}) \mid r \in \text{supp}(R)\}.$$

The root projection used by a single-row invariant interface is the partial function

$$\text{rootVal}_{a,\tau} : \text{Rows}_{a:\tau} \rightarrow \hat{D}_\tau^\perp, \quad \text{rootVal}_{a,\tau}(R) = r(a) \quad \text{when } \text{supp}(R) = \{r\}.$$

It is undefined for an empty or multi-row bag;  $\text{useIdSet}$  is the deliberate finite-Set projection for the general result bag.  $\text{Rows}_{\text{useId:ObjectId}}$  is the subcarrier in which every row has exactly the declared **useId: ObjectId** root field. For a finite bag  $R$ ,  $\text{supp}(R)$  is the Set of rows with positive multiplicity. The support operation discards row multiplicity only at this explicit Set projection; an absent, extra, or ill-typed root field is rejected rather than silently coerced.

**Where.**  $\text{ExecCypher}$  returns a finite bag of rows, and  $\text{useIdSet}$  is the only place where row multiplicity is removed.  $\text{supp}(R)$  denotes the rows with positive multiplicity, and each row must expose exactly one typed **useId** field. Missing, duplicate field names, extra root shapes, and ill-typed values are interface failures, not alternative identity semantics.

**Definition 7.3 (Exact stable violation-ID preservation)** Let the coordinated transformations produce

$$G = T_{\text{INSTANCE}}^M(I), \quad (q, \pi) = T_{\text{CONSTRAINT}}^M(C, e).$$

The successful chain is understood as

$$(C, e) \xrightarrow{T_{\text{BIND}}^M} b \xrightarrow{T_{\text{SEM}}^M} s \xrightarrow{T_{\text{NORM}}^M} n \xrightarrow{T_{\text{CQM}}^{M,\text{spec}}} p \xrightarrow{T_{\text{PLAN}}^{M,\text{spec}}} (q, \pi),$$

with  $\text{ValidNVA}(n)$  and  $q$  in the selected query profile. Preservation is

$$\text{ReturnedIds}(q, G, \pi) = \text{ViolIds}_{\text{OCL}}(e, C, M, I). \quad (\text{P})$$

Equation (P) requires both soundness (no spurious stable violation IDs) and completeness (no missing stable violation IDs).

**Where.**  $G = T_{\text{INSTANCE}}^M(I)$  is the canonical graph and  $(q, \pi)$  is the request generated from the same context-invariant pair  $(C, e)$ . Equation (P) is an equality of finite Sets of model-scoped identifiers; its two inclusions are the soundness and completeness directions used in the main theorem.

## 7.1 Assumption ledger

Table 4: Assumptions of the conditional preservation theorem.

| ID     | Requirement                                                                                                                                                                                                                                                                    | Why it is needed / failure if omitted                                                                                                                                           |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1     | $\text{Admit}_{\text{OCL}_{\text{val}}}(M, C, e)$ includes successful parsing, decoding, and profile admission.                                                                                                                                                                | Fixes the admitted representation and rejects ambiguous, unsupported, or ill-typed source terms before any semantic claim.                                                      |
| A2     | Context resolution and typed binding succeed, yielding $\Gamma_C = [\text{self} : \text{Obj}(C)]$ and a unique bound tree.                                                                                                                                                     | Makes model symbols, ownership, types, scopes, and the binding denotation well defined.                                                                                         |
| A3     | Evaluation uses the explicitly defined finite-Set validation semantics, typed bottoms, validation-truth collapse, and collection-boundary policy.                                                                                                                              | Fixes the source meaning; full OMG null/invalid and unrestricted collection kinds remain outside the theorem.                                                                   |
| A4     | $M \in \text{UML}_{WF}$ and $I \in \text{Inst}_{WF}(M)$ are finite and conforming.                                                                                                                                                                                             | Makes source ownership, typing, multiplicity, qualifiers, extents, and stable identity well defined.                                                                            |
| A5     | $\text{GraphAdequate}(M, I, G, \eta)$ holds through R1–R7; for the canonical encoder it is supplied by the schema/instance lemmas and Theorem 6.5.                                                                                                                             | Makes graph observers, object coverage, links, attributes, qualifiers, and context extents agree with the source state.                                                         |
| A6     | $n$ is a valid normal form, $T_{\text{CQM}}^{\text{spec}}(n)$ and $T_{\text{PLAN}}^{\text{spec}}$ are defined, and $\text{SpecPlanAdequacy}$ holds.                                                                                                                            | Supplies structural query adequacy; its constructor-local side conditions are split as A6-LR, A6-CQM (plan interface), and A6-ENV (parameter, alias, and scope correspondence). |
| A6-LR  | Every reachable NVA constructor satisfies the typed, scope, collection, bottom, freshness, and parameter side conditions $\text{Pre}_r$ in Table 3.                                                                                                                            | Supplies only syntactic/local well-formedness facts; the NVA-to-CQM semantic equations are conclusions of Lemma 5.3, not assumptions.                                           |
| A6-CQM | $T_{\text{CQM}}^{\text{spec}}$ and $T_{\text{PLAN}}^{\text{spec}}$ preserve result kind, aliases, parameters, typed-AST closure, and the root interface, and satisfy $\text{SpecPlanAdequacy}$ .                                                                               | Supplies structural plan-interface closure; concrete text emission remains an explicit refinement outside the theorem.                                                          |
| A6-ENV | Live graph environments and parameter maps correspond by typed values, scopes, and fresh aliases at every local constructor.                                                                                                                                                   | Prevents capture, alias collision, parameter substitution errors, and accidental cross-model matches.                                                                           |
| A7     | The generated query has exactly one public result column named <code>useId</code> , and every returned value has type <code>ObjectId</code> .                                                                                                                                  | Aligns executable rows with the theorem’s stable-identity observable; no node-returning interpretation is admitted.                                                             |
| A8     | $\text{ScalarClosed}(C, e, M, I)$ holds for every reachable scalar subevaluation and context object.                                                                                                                                                                           | Excludes overflow, non-finite arithmetic, inexact coercion, and zero-divisor behavior not covered by the scalar codec.                                                          |
| A9     | The selected Cypher/backend profile satisfies CY1–CY9 and $\text{BottomSeparated}(M, I, G, e, q, \pi)$ , including typed lookup, matching, filtering, existence, aggregation, control, correlation, branch composition, projection, parameter coverage, and bottom separation. | Relates the normative query model to the selected backend profile; behavior outside that profile and unrestricted values are outside the theorem.                               |

For exposition, the paper groups these assumptions as P1 (admission and source semantics: A1–A4 and A8), P2 (graph representation: A5 and R1–R7), P3 (typed query realization and result interface: A6–A7, including the A6-LR/A6-CQM/A6-ENV subobligations), and P4 (selected executor: A9). The normative constructor lemma derives the A6-LR semantic equations from their local side conditions; the main theorem does not treat “query realization” as one opaque preservation assumption. The grouping

must retain a cross-reference to this ledger.

## 8 Proof Development

### 8.1 Language-side preservation

**Lemma 8.1 (L1 — binding)** *For all  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $\Gamma \in \text{TypeEnv}$ ,  $\tau$ ,  $e \in \text{OCLExpr}_\tau(M, \Gamma)$ ,  $b \in \text{BoundExpr}_\tau(M, \Gamma)$ , and  $\rho \in \text{Env}_I(\Gamma)$ , if A1 holds and  $T_{\text{BIND}}^M(e) = b$ , then*

$$\llbracket e \rrbracket_{\text{OCL}}^{M, I, \rho} = \llbracket b \rrbracket_{\text{src}}^{M, I, \rho}.$$

*Proof.* Proceed by induction on  $M; \Gamma \vdash e \Downarrow b : \tau$ . Variables use the unique declaration recorded in  $\Gamma$ ; literals are unchanged. Attribute and navigation cases use the induction hypothesis for the receiver and the resolved owner/role identity, so both sides invoke the same source observer. Composite Boolean, scalar, Set, and control cases follow by congruence. In a let or iterator case, freshness permits extending both environments with the same value; the body induction hypothesis then applies. Type and coercion cases use the recorded conformance witness. These cases exhaust the grammar in Section 4. Parse, ambiguity, scope, or typing failure has no derivation and is outside A1.  $\square$

**Lemma 8.2 (L2 — semantic abstraction)** *For all  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $\Gamma \in \text{TypeEnv}$ ,  $\tau$ ,  $b \in \text{BoundExpr}_\tau(M, \Gamma)$ ,  $s \in \text{OVA}_\tau(M, \Gamma)$ , and  $\rho \in \text{Env}_I(\Gamma)$ , if  $b$  is well typed and  $T_{\text{SEM}}^M(b) = s$ , then*

$$\llbracket b \rrbracket_{\text{src}}^{M, I, \rho} = \llbracket s \rrbracket_{\text{src}}^{M, I, \rho}$$

.

*Proof.* Use structural induction on  $b$ . The constructor map in Section 5 is identity-on-denotation for atoms and resolved accesses. Every composite OVA constructor applies the same semantic operation to recursively transformed children, so the induction hypotheses and congruence discharge it. Let and iterator cases additionally use identical symbol identifiers and corresponding environment extension. Erased surface spelling is not semantically observed; resolved attribute, role, class, multiplicity, and type witnesses are retained.  $\square$

**Lemma 8.3 (L3 — normalization)** *For all  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $\Gamma \in \text{TypeEnv}$ ,  $\tau$ , well-typed  $s \in \text{OVA}_\tau(M, \Gamma)$ ,  $n \in \text{NVA}_\tau(M, \Gamma)$ , and  $\rho \in \text{Env}_I(\Gamma)$ , if  $T_{\text{NORM}}^M(s) = n$  terminates in a normal form, then of the same type and scope, and*

$$\llbracket s \rrbracket_{\text{src}}^{M, I, \rho} = \llbracket n \rrbracket_{\text{src}}^{M, I, \rho}$$

.

*Proof.* Induct on the well-founded measure  $\mu$  from Section 5. For a structural lift, every child has a smaller measure and the induction hypotheses compose by congruence. For a derived rewrite, the displayed local equations give extensional equivalence. As a bottom-sensitive case, forAll and not-exists-not have the same result when a predicate is false, when every predicate is true, and when neither decisive value exists but a predicate is bottom. Emptiness follows from existence over the finite-Set view; membership rules follow from extensional equality after the declared coercion. All rewrites retain result type and lexical symbol identifiers. Each step strictly decreases  $\mu$ , so the process terminates in a normal form and the finite equality chain yields the stated denotation equality.  $\square$

Together L1–L3 give

$$\llbracket e \rrbracket_{\text{OCL}} = \llbracket b \rrbracket_{\text{src}} = \llbracket s \rrbracket_{\text{src}} = \llbracket n \rrbracket_{\text{src}}. \quad (\text{L123})$$

### 8.2 Representation-side preservation

**Lemma 8.4 (L4 — graph evaluation)** *For all  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $G \in \text{PG}(T_{\text{SCHEMA}}(M))$ ,  $\eta : O \rightarrow V_{\text{obj}}(G)$ ,  $\Gamma \in \text{TypeEnv}$ ,  $\tau$ ,  $n \in \text{NVA}_\tau(M, \Gamma)$ ,  $\rho \in \text{Env}_I(\Gamma)$ , and  $\gamma \in \text{Env}_G(\Gamma)$ , assume A2–A4,*

$$\text{GraphAdequate}(M, I, G, \eta).$$

*Assume also  $\text{GraphWF}(T_{\text{SCHEMA}}(M), G)$  and  $\rho \sim_\eta \gamma$ ; then*

$$\text{encodeValue}_\tau(\llbracket n \rrbracket_{\text{src}}^{M, I, \rho}) = \llbracket n \rrbracket_{\text{graph}}^{T_{\text{SCHEMA}}(M), G, \gamma}.$$

*Proof.* By structural induction on  $n$ . Variables use  $\rho \sim_\eta \gamma$ ; literals use value encoding; attributes, navigation, and extents use R4, R5–R6, and R7. Scalar/Boolean operations commute by A3, while if/let preserve the selected branch and related environments. Finite-Set constructors and iterators use pointwise encoding, extensionality, the element bijection, and the same three-valued tables; collection bottom remains separated. Type tests and casts use R3 and the bottom-safe cast clause. These cases exhaust the normalized grammar of Section 4.5.  $\square$

### 8.3 Executable realization

For  $v \in V_C(G)$  let  $\gamma_v = [\mathbf{self} \mapsto v]$  and

$$P_G(v) = \text{vtrue}_G(\llbracket n \rrbracket_{\text{graph}}^{T_{\text{SCHEMA}}(M), G, \gamma_v}).$$

**Lemma 8.5 (L5 — executable Cypher realization)** *Let  $M \in \text{UML}_{WF}$ ,  $I \in \text{Inst}_{WF}(M)$ ,  $G \in \text{PG}(T_{\text{SCHEMA}}(M))$ , and let  $n \in \text{AdmittedNVA}$ . Under A5, A6-LR, A6-CQM, A6-ENV, and A7–A9, suppose*

$$\text{Realize}^M(n) = (q, \pi).$$

*with  $p = T_{\text{CQM}}^M(n)$  and  $\text{SpecPlanAdequacy}(M, \text{ctx}_{\text{NVA}}(n), n, p, q, \pi, G)$ . Then*

$$\text{ReturnedIds}(q, G, \pi) = \{\text{prop}(v, p_{id}^M) \mid v \in V_C(G), P_G(v) = \text{false}\}.$$

*Proof.* Induct over the admitted NVA derivation. For each constructor, apply Lemma 5.3 and the child induction hypotheses; A6-LR supplies only its typed syntactic side conditions. A6-ENV supplies the same parameter values, fresh aliases, and corresponding live graph context; therefore let, iterator, correlated existence, and branch cases compose. The CY equation from A9 replaces evaluation of the resulting CQM plan by execution of rendered Cypher. At the invariant root, the wrapper matches exactly  $V_C(G)$ , filters by not-validation-true, and A8 projects only Set-distinct  $\text{prop}(v, p_{id}^M)$  values. This is precisely the Set displayed in the conclusion.  $\square$

The proof is conditional because A6-CQM relates a concrete plan construction to the normative rules and A9 fixes the selected executor profile. A6-LR no longer assumes denotation preservation: it supplies the local typing, scope, collection, bottom, freshness, and parameter conditions used by the proved constructor induction.

### 8.4 Main theorem

**Theorem 8.6 (Conditional stable violation-ID preservation)** *Let*

$$M \in \text{UML}_{WF}, \quad I \in \text{Inst}_{WF}(M), \quad C \in \mathcal{C}, \quad e \in \text{Inv}_{val}(M, C),$$

*and let*

$$G = T_{\text{INSTANCE}}^M(I), \quad (q, \pi) = T_{\text{CONSTRAINT}}^M(C, e).$$

*If A3, A5, A6-LR, A6-CQM, A6-ENV, and A7–A9 hold (with A1 and A2 supplied by the displayed memberships, and A4 supplied by Lemmas 6.3 and 6.4), then*

$$\text{ReturnedIds}(q, G, \pi) = \text{ViolIds}_{\text{OCL}}(e, C, M, I).$$

*Proof.* The pipeline supplies  $b, s, n$ . Lemmas 6.3 and 6.4 discharge A4 for the canonical transformations, and L1–L3 give (L123). Canonical encoder adequacy supplies R1–R7 and hence the remaining representation premise of L4. L5 characterizes the returned IDs.

For soundness, choose  $x \in \text{ReturnedIds}(q, G, \pi)$ . L5 gives  $v \in V_C(G)$  with graph validation false and  $x = \text{prop}(v, p_{id}^M)$ . R2 and R7 yield a unique  $o \in O_C^I$  with  $v = \eta(o)$ ; R1 yields  $x = \text{id}_I(o)$ . L1–L4 and the commuting Boolean view make the source evaluation at  $o$  validation-false. Thus  $x \in \text{ViolIds}_{\text{OCL}}$ .

For completeness, choose a source-violating  $o \in O_C^I$  and let  $x = \text{id}_I(o)$ . R7 puts  $\eta(o)$  in  $V_C(G)$ ; L1–L4 and the Boolean-view equation make its graph evaluation validation-false. L5 returns its ID, and R1 identifies that value with  $x$ . Both inclusions establish Set equality.  $\square$

**Consequence.** The theorem establishes exact stable-ID preservation for the admitted finite-Set profile and the canonical graph encoder, provided the displayed local realization and selected-backend obligations hold. It does not establish correctness for arbitrary OCL, arbitrary Cypher, or arbitrary graph encodings; any concrete execution realization requires a separate refinement certificate.

## 8.5 Dependency and obligation map

P-01/P-02 discharge L1, P-03 discharges L2, P-04 discharges L3, P-07 supplies encoder adequacy and L4, P-05/P-06 supply L5, and P-08 is the two-inclusion main theorem. Equivalently, PC-T0–PC-T6 denote adequacy, L1–L5, and their composition. Supporting families M1–M5, G1–G4, C1–C6, and TXT respectively cover source closure, graph/value correspondence, query composition, and text realization; they refine rather than replace the stated local premises.

## 9 Formal Status and Trusted Boundary

The obligation inventory contains 24 rows: 22 discharged under their stated premises, one partial row for universal primitive and realization coverage, and one publication-only row outside correctness. The formal derivations cover finite-Set transport, typed encoding, normalization, constructor coverage, graph/value correspondence, and the two stable-ID inclusions. They establish the mathematical result under its assumptions; they do not assert a universal property of a particular software or storage system.

At the mechanization boundary, Lean kernel-checks 58 formal declarations, including auxiliary lemmas supporting selected L1–L4 finite-Set, value-encoding, normalization, graph-correspondence, and case-study obligations. This count is a mechanization inventory, not a claim that L5, the main theorem, or the Java/Neo4j refinement has been proved in Lean.

L1–L4 are mathematical inductions over typed trees, rewrite equations, finite Sets, and graph clauses. L5 and the main theorem are conditional on the LR/CY and backend premises. Differential observations are empirical only. A3, A5, A6-LR/A6-CQM/A6-ENV, and A7–A9 remain open whenever their displayed obligations have not been discharged.

The open composition boundary is the typed chain

$$\text{AdmittedNVA} \longrightarrow \text{TypedCQM} \longrightarrow \text{CypherText} \longrightarrow \text{DriverRequest} \longrightarrow \text{Rows}.$$

The first two arrows are governed by the local LR/CY equations and the declared result interface. The final arrows are selected semantic premises about row production and bottom separation; they are not universal theorems. Finite observations can support those premises for a declared profile, but do not replace the corresponding refinement argument. Only the semantic and mathematical interfaces are within the scope of this report.

## 10 Evaluation Semantics and Evidence

The three evaluation questions are subquestions of the paper’s main research question:

**EQ1 — admission.** Are in-profile cases accepted and designated non-profile cases rejected at the intended boundary?

**EQ2 — agreement.** For executed cases, do generated queries return exactly the stable violation IDs reported by the source oracle?

**EQ3 — boundary evidence.** Which assumptions and proof gates receive finite evidence, and which remain open premises?

For reference Set  $V_{\text{ref}}$  and target Set  $V_{\text{Cy}}$ , the differential observations are

$$\text{missing} = V_{\text{ref}} \setminus V_{\text{Cy}}, \quad \text{spurious} = V_{\text{Cy}} \setminus V_{\text{ref}}.$$

Agreement requires both Sets to be empty. The reference evaluator and the mathematical OCL interpretation are related by a declared oracle relation for the finite observations.

Table 5: Claim-oriented evidence in the reported evaluation.

| Evidence family           | Result          | Interpretation                                                                    |
|---------------------------|-----------------|-----------------------------------------------------------------------------------|
| Cross-layer feature cases | 47/47           | Exact stable violation-ID agreement in the checked profile                        |
| Case-study theorem-scope  | 10/10           | Satisfies the source theorem profile                                              |
| Case-study out-of-scope   | 13/13           | Finite implementation agreement only                                              |
| Admission boundary        | 39/39           | Expected deterministic rejection before execution                                 |
| Runtime/property probes   | 10/10; 64/64    | Selected real-Neo4j tests and generated-property cases                            |
| Backend observations      | 9/9             | Finite evidence for the selected Cypher/Neo4j profile                             |
| Mutation gates            | 7/7; 7/7; 20/20 | Machine-contract, Lean proof-gate, and compiler mutants killed                    |
| Lean-checked declarations | 58              | Formal declarations including auxiliary L1–L4 lemmas, not end-to-end verification |

**Why 13/23 and 39/39 mean different things.** The 13 out-of-profile case-study observations produce the same stable violation-ID Sets in the reference and target evaluations on the sampled states. Their “success” is therefore finite semantic agreement outside at least one theorem-profile premise. By contrast, the 39 negative-admission cases are constructed to be illegal, unbound, ill typed, capture-prone, or unsupported. Their expected success condition is deterministic rejection before target execution. The two rows test different boundaries and must never be summed as if they were homogeneous positive cases.

**Representative boundary perturbation.** For invariant `Adult: self.age >= 18`, let the source state contain `research_minor` with age below 18 and `research_boundary` with age exactly 18. The correct source violation Set is `{research_minor}`. Perturbing the generated comparison from `>=` to `>` produces `{research_minor, research_boundary}`. The differential oracle reports `research_boundary` as spurious and rejects the perturbation. The diagnosis is loss of the inclusive boundary during query generation.

These observations provide finite evidence for the stated cross-layer agreements and proof gates. They neither discharge every A4–A9 premise nor establish the universal conditional theorem.

## 11 Evidence Protocol

Quantitative statements refer to a declared evaluation protocol. A complete record specifies the source configuration, admitted profile, model and instance, each transformation result, the source and target semantic oracles, the observed result rows, and the measurement totals. For every case the protocol checks admission and typing, constructs the graph and query representations, compares the two stable-ID Sets, and records rejection, failure, or incompleteness separately from successful agreement. Only cases whose premises are explicit are included in a quantitative total. Operational details are outside the mathematical claim.

The clean capture used USE, Neo4j Enterprise with Cypher, the Java driver, a JDK, and Lean. Its manifest is `clean-capture.tsv`; the accompanying case-study record is `medical-carrental-experiment.md`. The capture reports ten real-Neo4j tests with zero failure, error, or skip and records the normative, realization-specific, and empirical inputs. The principal reproduction entry points are

```
powershell -File verification/scripts/invoke-clean-evidence-capture.ps1
-RunRuntime
mvn -pl neo4j-tgg clean test
lake build (from verification/lean)
```

Compilation failure, EXPLAIN failure, execution failure, or a skipped runtime case is recorded as non-agreement; no such outcome is converted into a passing observation.

## 12 Limitations and Claim Policy

The formal result is universal only over values satisfying its quantified domains and assumptions. The empirical evidence is finite and tied to the declared evaluation protocol. Consequently:

Out-of-profile agreement does not extend the theorem, and rejection validates an admission boundary rather than post-execution equivalence. Auxiliary results do not establish concrete execution refinement; the canonical encoder does not cover arbitrary graph layouts; selected Cypher observations do not define behavior outside the declared backend profile. Failure of admission, adequacy, constructor coverage, `SpecPlanAdequacy`, `ScalarClosed`, `BottomSeparated`, or CY1–CY9 yields no execution claim. No performance, full-OCL completeness, or scalability claim is made.

The strongest justified wording is therefore: the construction has a conditional exact stable violation-ID preservation theorem for the admitted profile, and the declared evaluation provides finite cross-layer evidence for selected assumptions and boundary conditions.

## A Admitted Surface Summary

The 47 coverage units are: `self` and lexical variables; four literal kinds; attribute, forward/reverse/qualified navigation, and `allInstances`; five Boolean operators; six comparisons; four arithmetic operators; `if` and `let`; five predicate iterators plus scalar `collect`; four Set relations; `size`, `isEmpty`, and `notEmpty`; two type operations; Set literal, `union`, `intersection`, `asSet`, and `isUnique`. Their typing families are T-VAR, T-LIT, T-ATTR, T-NAV, T-ALL, T-BOOL, T-COMP, T-ARITH, T-IF, T-LET, T-ITER, T-COLLECT, T-SET, T-SETOP, and T-TYPE. Sections 4 and 4.5 give the denotations; L1–L4 cover the same families by structural induction. Admission still requires successful binding and typing, so the summary does not admit arbitrary untyped spellings.

## B NVA-to-CQM Rule Catalog

Table 6: Total constructor catalog for admitted NVA.

| Rule: NVA constructor $\rightarrow$ CQM constructor; obligation                              | Rule: NVA constructor $\rightarrow$ CQM constructor; obligation                                 |
|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>SPS-VARIABLE:</b> NvaVariable $\rightarrow$ VariablePlan;<br>LR-Variable                  | <b>SPS-LITERAL:</b> NvaLiteral $\rightarrow$ LiteralPlan; LR-Literal                            |
| <b>SPS-SET:</b> NvaSetLiteral $\rightarrow$ SetPlan; LR-SetLiteral                           | <b>SPS-ATTRIBUTE:</b> NvaAttribute $\rightarrow$ AttributeAccessPlan;<br>LR-Attribute           |
| <b>SPS-NAV-ONE:</b> NvaNavigationOne $\rightarrow$<br>NavigationAccessPlan; LR-NavigationOne | <b>SPS-NAV-MANY:</b> NvaNavigationMany $\rightarrow$<br>NavigationAccessPlan; LR-NavigationMany |
| <b>SPS-VIEW-SET:</b> NvaViewSet $\rightarrow$ CallPlan;<br>LR-CollectionView                 | <b>SPS-ALL-INSTANCES:</b> NvaAllInstances $\rightarrow$ CallPlan;<br>LR-AllInstances            |
| <b>SPS-NOT:</b> NvaNot $\rightarrow$ UnaryPlan; LR-Boolean                                   | <b>SPS-AND:</b> NvaAnd $\rightarrow$ BinaryPlan; LR-Boolean                                     |
| <b>SPS-OR:</b> NvaOr $\rightarrow$ BinaryPlan; LR-Boolean                                    | <b>SPS-COMPARE:</b> NvaCompare $\rightarrow$ BinaryPlan; LR-Scalar                              |
| <b>SPS-ARITH:</b> NvaArithmetic $\rightarrow$ BinaryPlan; LR-Scalar                          | <b>SPS-COERCE:</b> NvaCoerce $\rightarrow$ CallPlan; LR-Coercion                                |
| <b>SPS-IF:</b> NvaIf $\rightarrow$ IfPlan; LR-Conditional                                    | <b>SPS-LET:</b> NvaLet $\rightarrow$ LetPlan; LR-Let                                            |
| <b>SPS-EXISTS:</b> NvaExists $\rightarrow$ IteratorPlan; LR-Quantifier                       | <b>SPS-SELECT:</b> NvaSelect $\rightarrow$ IteratorPlan; LR-Filter                              |
| <b>SPS-COLLECT:</b> NvaCollect $\rightarrow$ IteratorPlan; LR-Collect                        | <b>SPS-UNIQUE:</b> NvaIsUnique $\rightarrow$ IteratorPlan; LR-IsUnique                          |
| <b>SPS-SET-REL:</b> NvaSetRelation $\rightarrow$ CallPlan;<br>LR-SetRelation                 | <b>SPS-SET-COMB:</b> NvaSetCombination $\rightarrow$ CallPlan;<br>LR-SetCombination             |
| <b>SPS-AS-SET:</b> NvaAsSet $\rightarrow$ CallPlan; LR-AsSet                                 | <b>SPS-COUNT:</b> NvaCount $\rightarrow$ CallPlan; LR-Cardinality                               |
| <b>SPS-KIND-OF:</b> NvaTypeKindOf $\rightarrow$ CallPlan;<br>LR-TypeAccess                   | <b>SPS-CAST:</b> NvaCast $\rightarrow$ CallPlan; LR-TypeAccess                                  |

## References

- [1] Object Management Group, *Object Constraint Language*, OMG Document formal/2014-02-03, 2014.
- [2] M. Gogolla, F. Büttner, and M. Richters, “USE: A UML-based specification environment for validating UML and OCL,” *Science of Computer Programming*, vol. 69, nos. 1–3, pp. 27–34, 2007.
- [3] N. Francis et al., “Cypher: An evolving query language for property graphs,” in *Proceedings of SIGMOD*, 2018, pp. 1433–1445.
- [4] J. Marton, G. Szárnyas, and D. Varró, “Formalising openCypher graph queries in relational algebra,” in *Proceedings of ADBIS*, 2017, pp. 182–196.